

Deep Generative Models for Vision

**Variational AutoEncoders, Generative
Adversarial Networks and Diffusion models**

Dr.Srijith P.K.
Dept of CSE and Dept of AI
Indian Institute of Technology Hyderabad

Outline

Introduction

Generative Modelling

Latent Variable Models

Variational AutoEncoders

Generative Adversarial Networks

Diffusion Models

Generative artificial intelligence (generative AI, GenAI,^[1] or GAI) is artificial intelligence capable of generating text, images, videos, or other data using generative models,^[2]

Generative AI models learn the patterns and structure of their input training data and then generate new data that has similar characteristics



Generated by Nano Banana

Which of these faces are real ?



Which of these faces are real ?



1. AI female
2. Human male
3. Human male
4. AI female
5. AI male
6. Human male
7. AI male
8. AI male
9. Human female
10. Human male

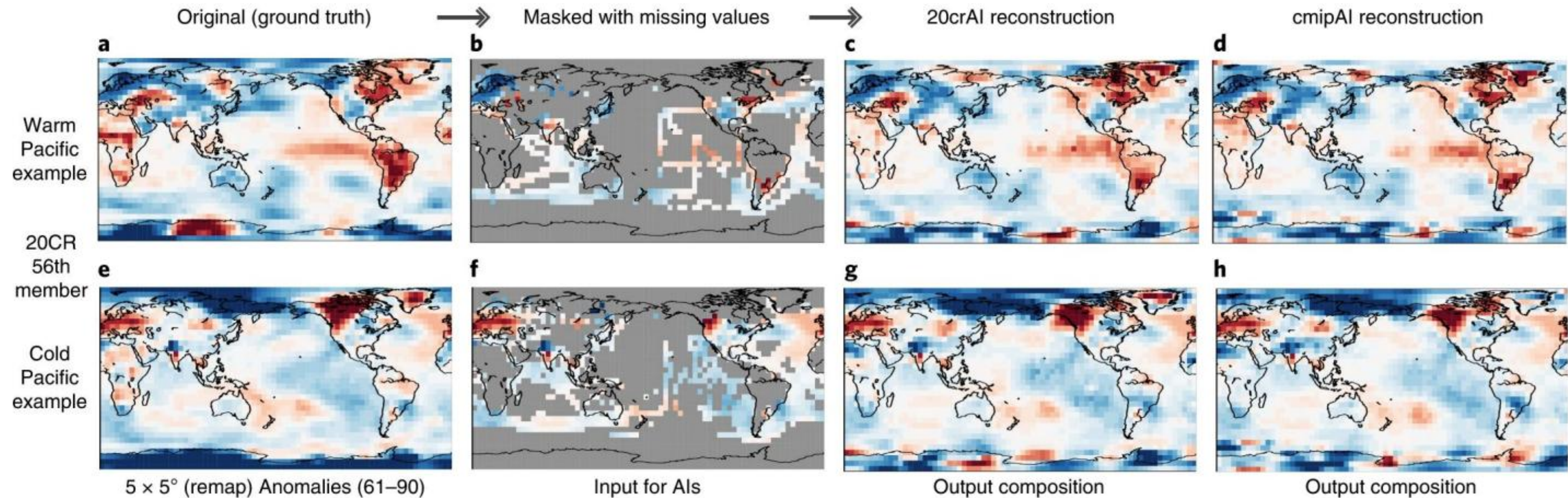
Denoising/ Image inpainting



nvidia.com/research/inpainting/

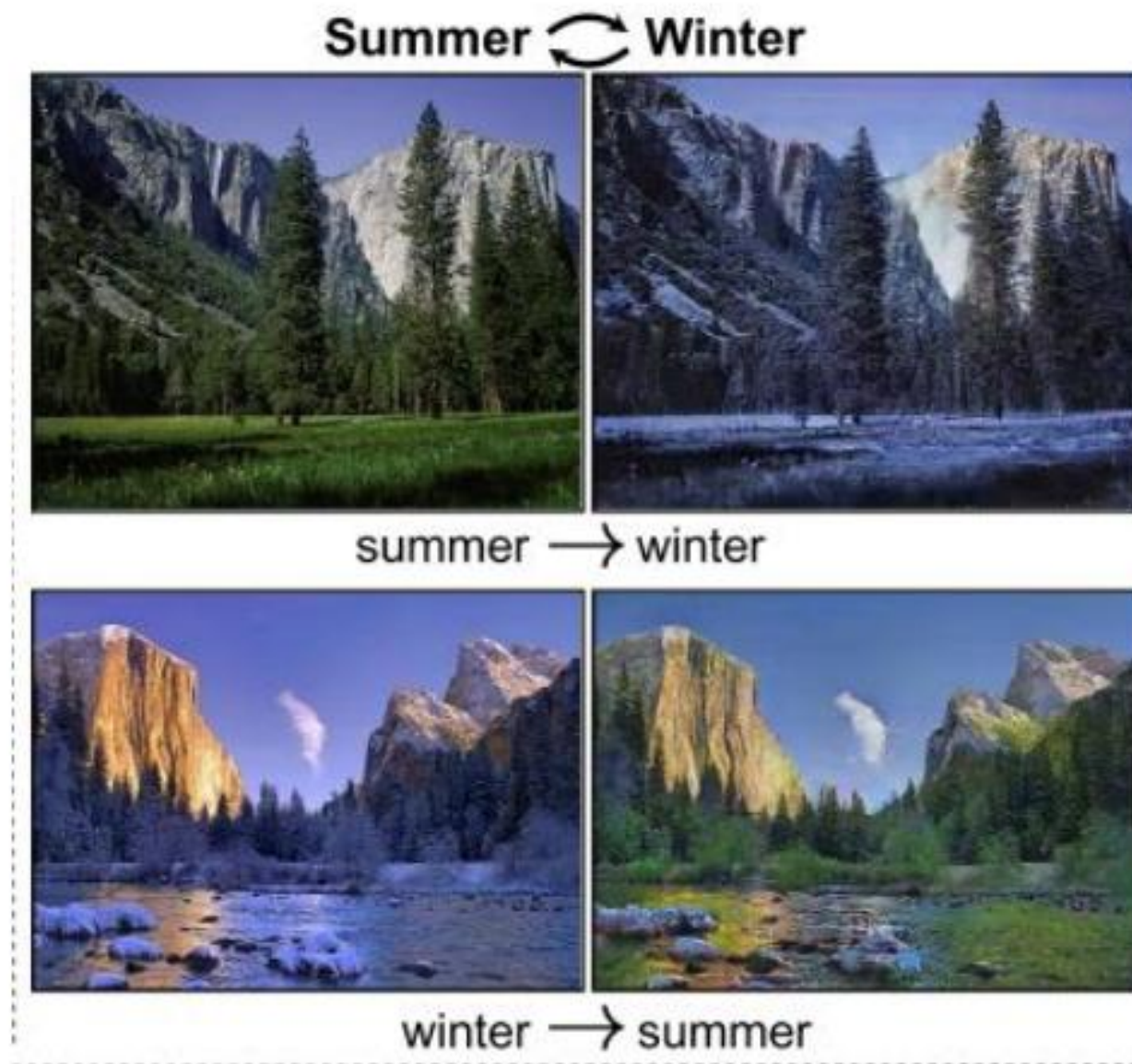
Generative models can be used to denoise or inpaint missing data.

Generative models used to reconstruct observational gaps in historical temperature measurements.



Domain-to-domain translation

CycleGAN, Zhu et al, [arXiv:1703.10593](https://arxiv.org/abs/1703.10593)



Generative models can be used to “translate” data from domain-to-another.

Schmidt et al. [arXiv:1905.03709](https://arxiv.org/abs/1905.03709)

“Visualizing the Consequences of Climate Change Using Cycle-Consistent Adversarial Networks”



Super-resolution

SRGAN, Ledig et al, [arXiv:1609.04802](https://arxiv.org/abs/1609.04802)



bi-cubic

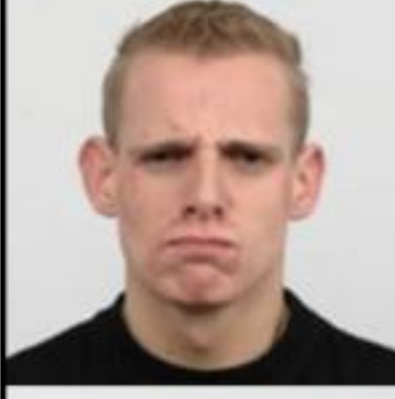


SR-GAN

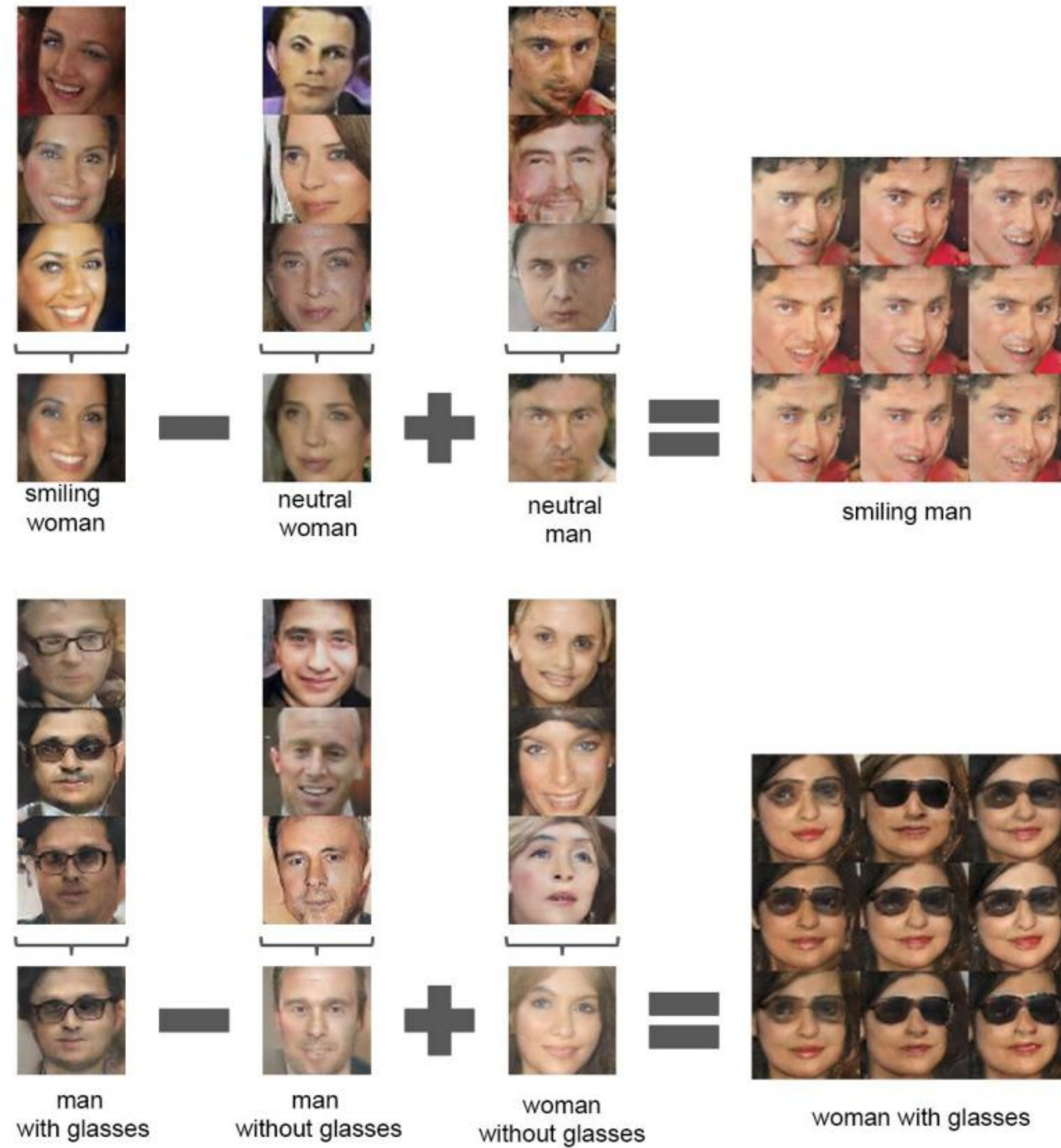


Original

Missing data imputation

Angry	Contemptuous	Disgusted	Fearful	Happy	Surprised	Sad	Neutral	Proposed	CycleGAN	StarGAN	Target
								 0.0197/0.794	 0.0301/0.732	 0.0306/0.698	
								 0.0172/0.802	 0.0362/0.694	 0.0397/0.683	
								 0.0211/0.778	 0.0350/0.682	 0.0420/0.606	
								 0.0204/0.776	 0.0352/0.698	 0.0395/0.652	

Learning useful representations



Supervised Learning

Given observed data $x_1, x_2, \dots, x_n$ and labels $y_1, y_2, \dots, y_n$, learn a mapping from $X \rightarrow Y$, i.e. model the conditional distribution

$$P(Y|X = x)$$

Examples: classification, regression, object detection, segmentation, etc.

Supervised Learning

Given observed data $x_1, x_2, \dots, x_n$ and labels $y_1, y_2, \dots, y_n$, learn a mapping from $X \rightarrow Y$, i.e. model the conditional distribution

$$P(Y|X = x)$$

Examples: classification, regression, object detection, segmentation, etc.

Unsupervised Learning

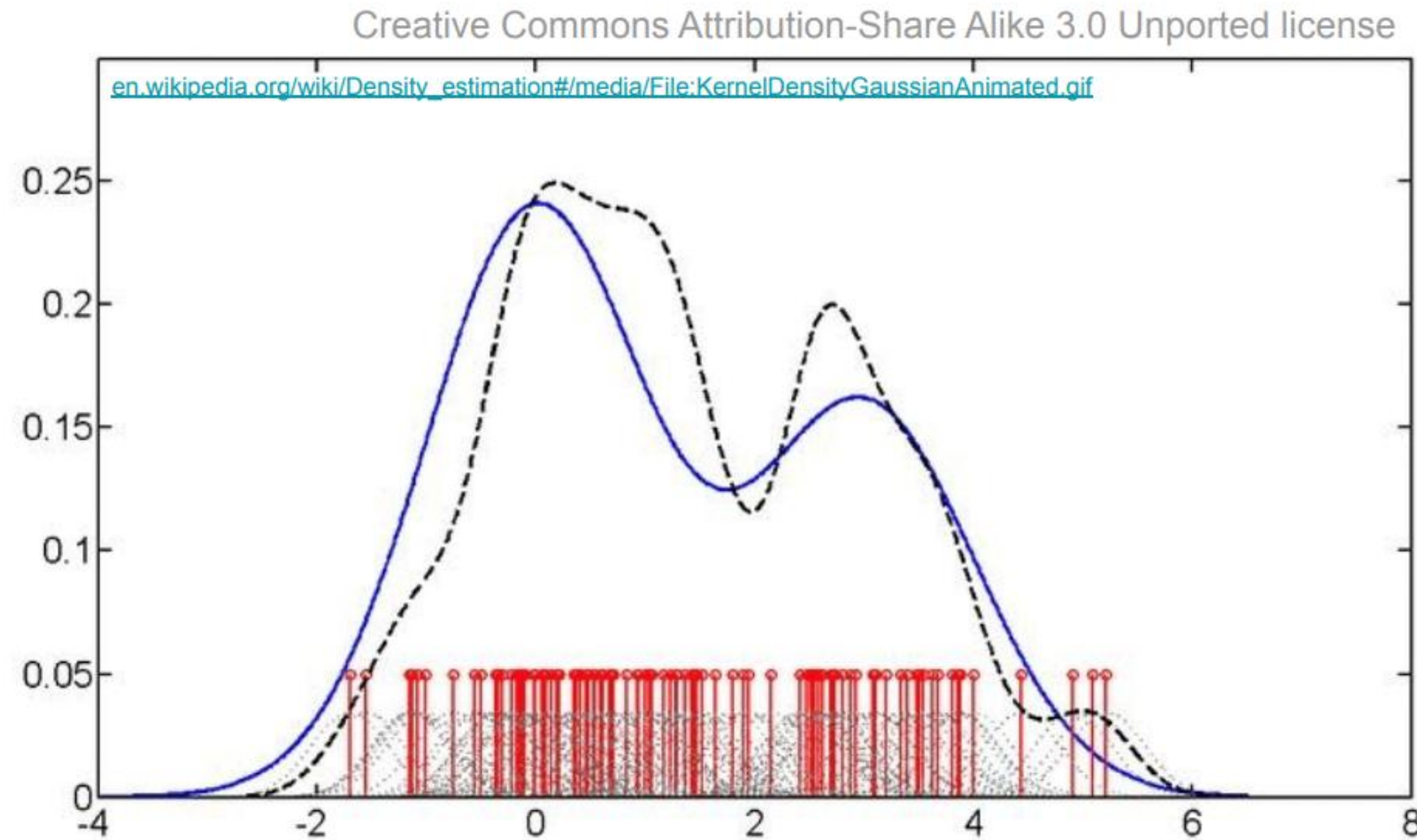
Given observed data $x_1, x_2, \dots, x_n$ learn the underlying structure of the data, i.e. model the data distribution

$$P(x)$$

Examples: dimensionality reduction, data generation, data imputation, data completion, etc.

Density estimation

An example of density estimation of observed data (red) using Kernel Density Estimation (dashed black). True distribution is a mixture of two-gaussians (blue).



Deep generative models are density estimators $P_{model}(x)$ of high-dimensional data $P_{data}(x)$ (e.g. images)

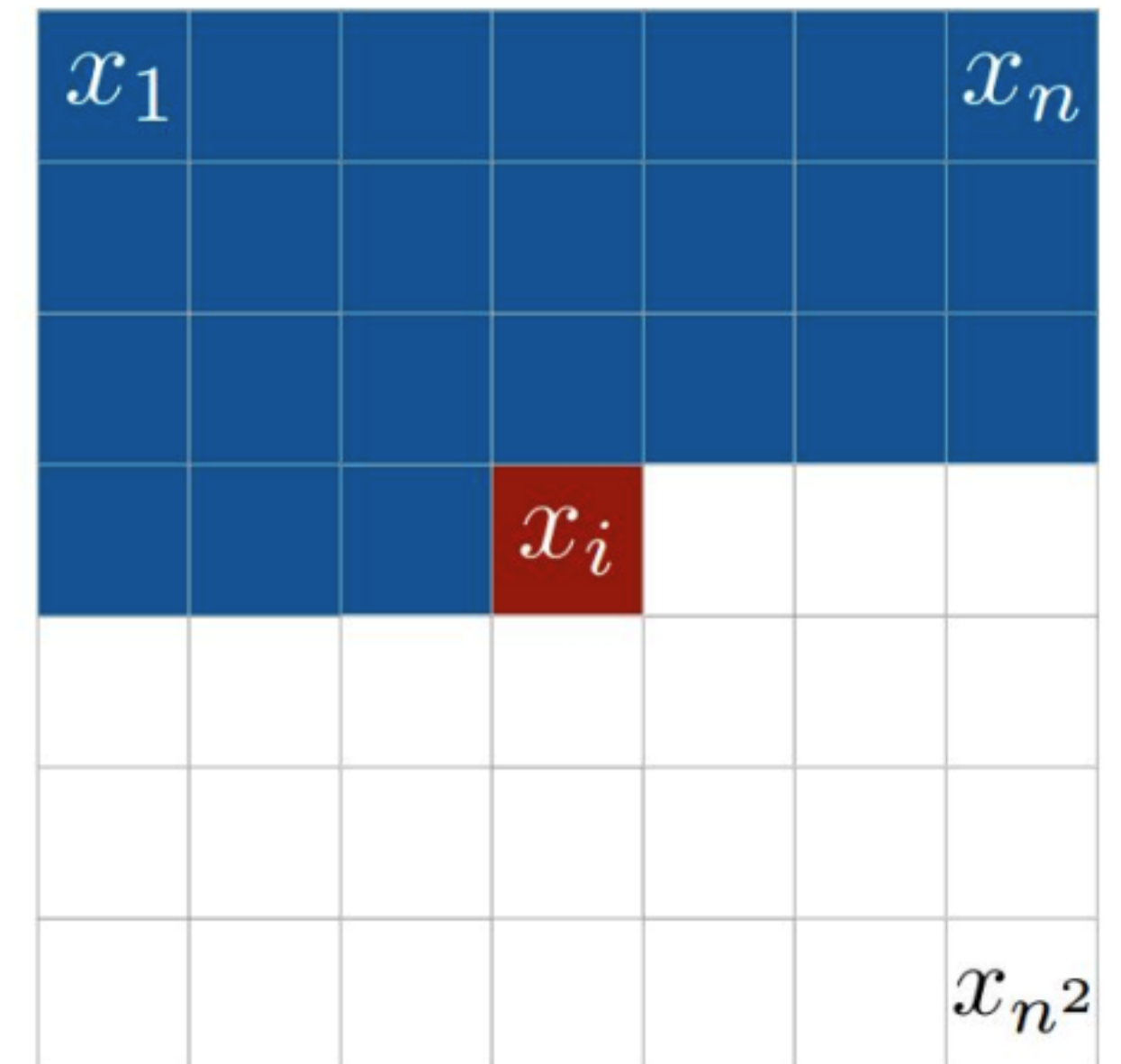
Decompose the data and model it auto-regressively. For example, the likelihood of an image $p(x)$ can be decomposed in terms of the other 1-d distributions:

$$p(x) = \prod_{i=1}^{n^2} p(x_i | x_1, \dots, x_{i-1})$$

likelihood of image X

likelihood of the i^{th} pixel

Figure credit: [H. Sharma](#)



Then use maximum-likelihood to maximize the likelihood of training data.

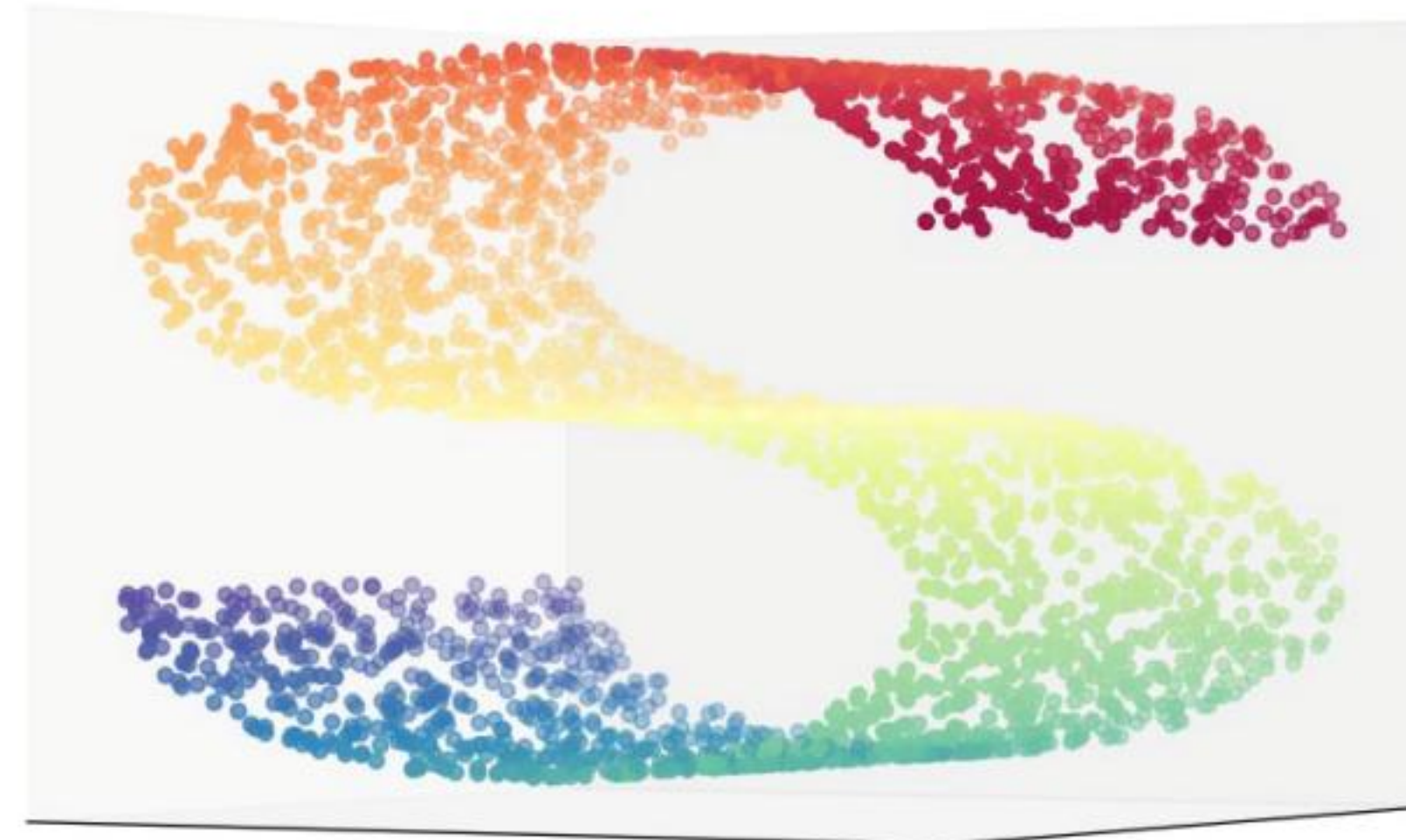
- Language modelling following auto regressive modelling
- Poor Global Structure Modeling

Latent Variables to model Data

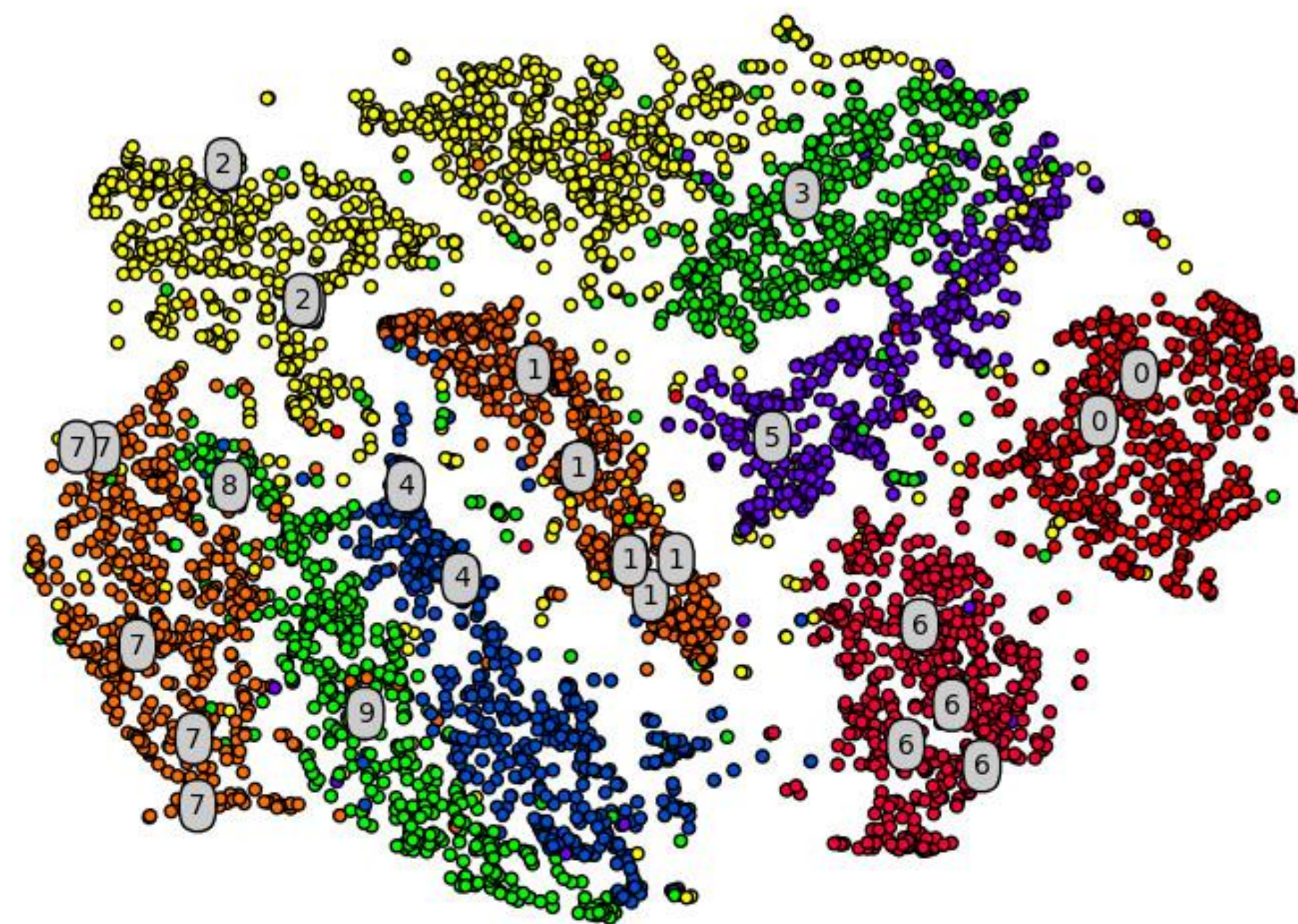
real-world high-dimensional data lie on low-dimensional manifolds embedded within the high-dimensional space.

We want to discover the complicated features of this lower-dimensional manifold, often with the downstream task in mind.

- Recall Text Embeddings

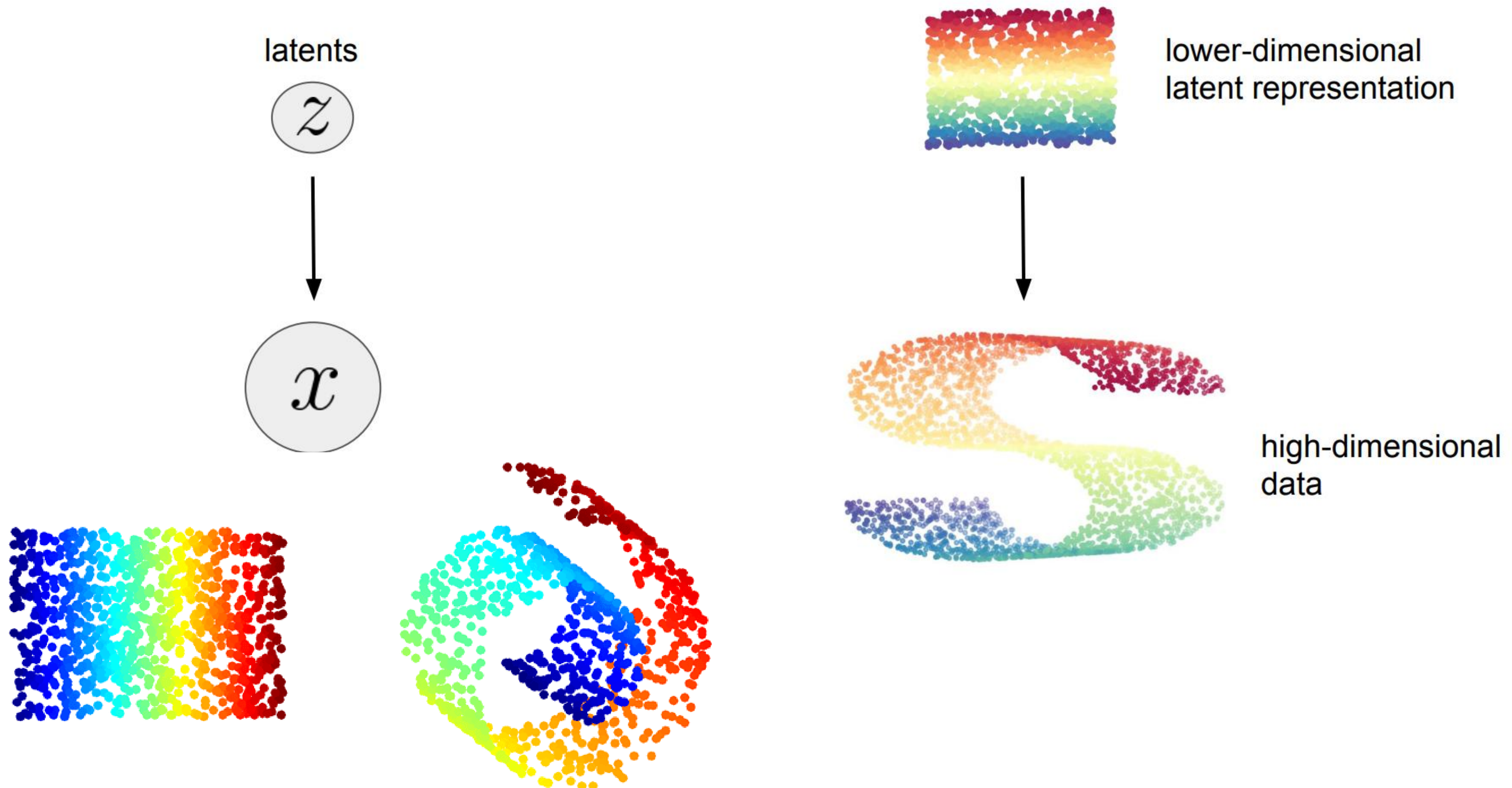


TSNE for MNIST digits



Latent variable models

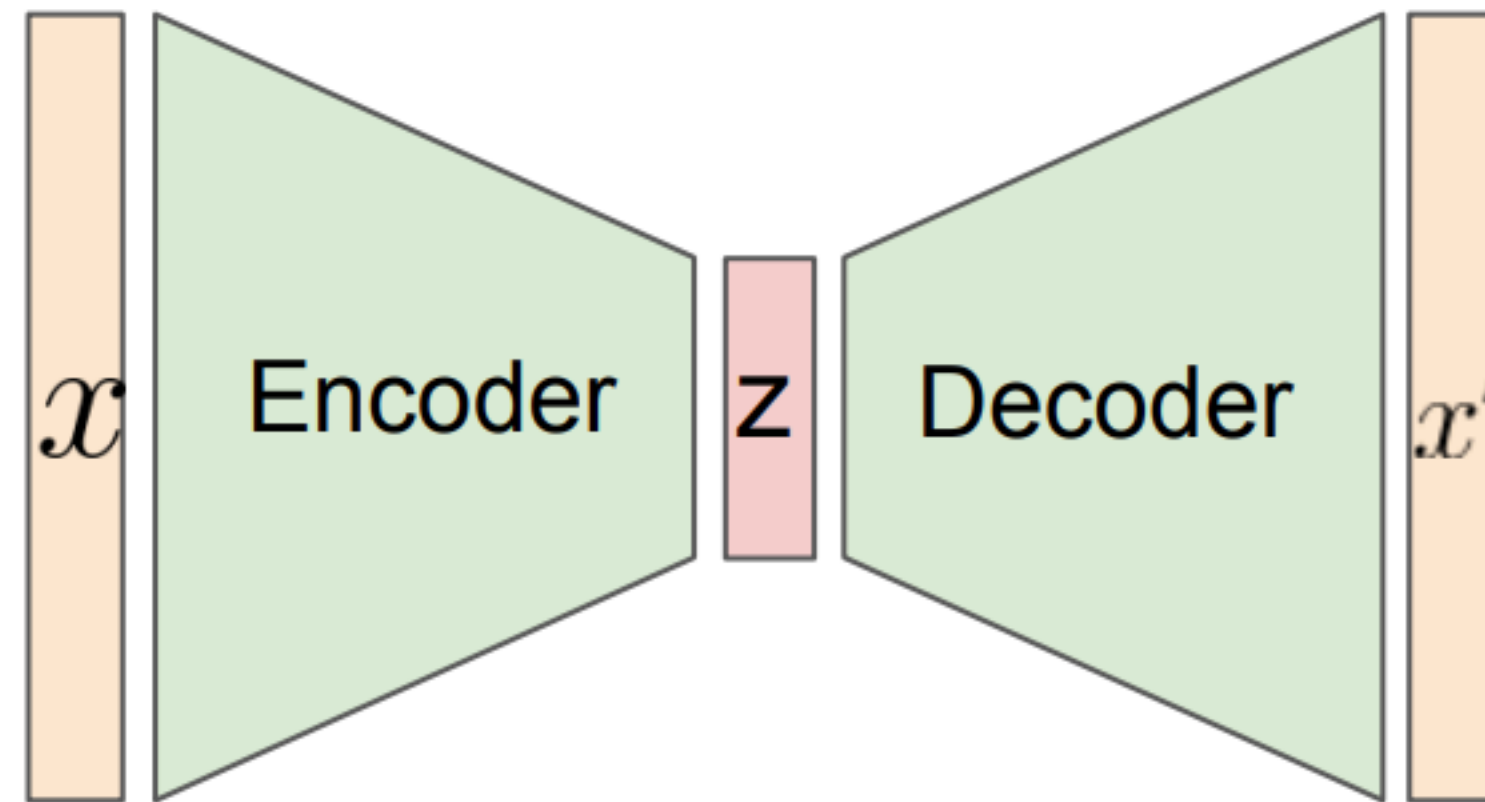
Learn a mapping between a simpler (latent) manifold and the real data-manifold.



Variational AutoEncoders

AutoEncoders

Learn to encode data into a lower-dimensional latent representation by training the model to reconstruct the data passing through an information bottleneck.

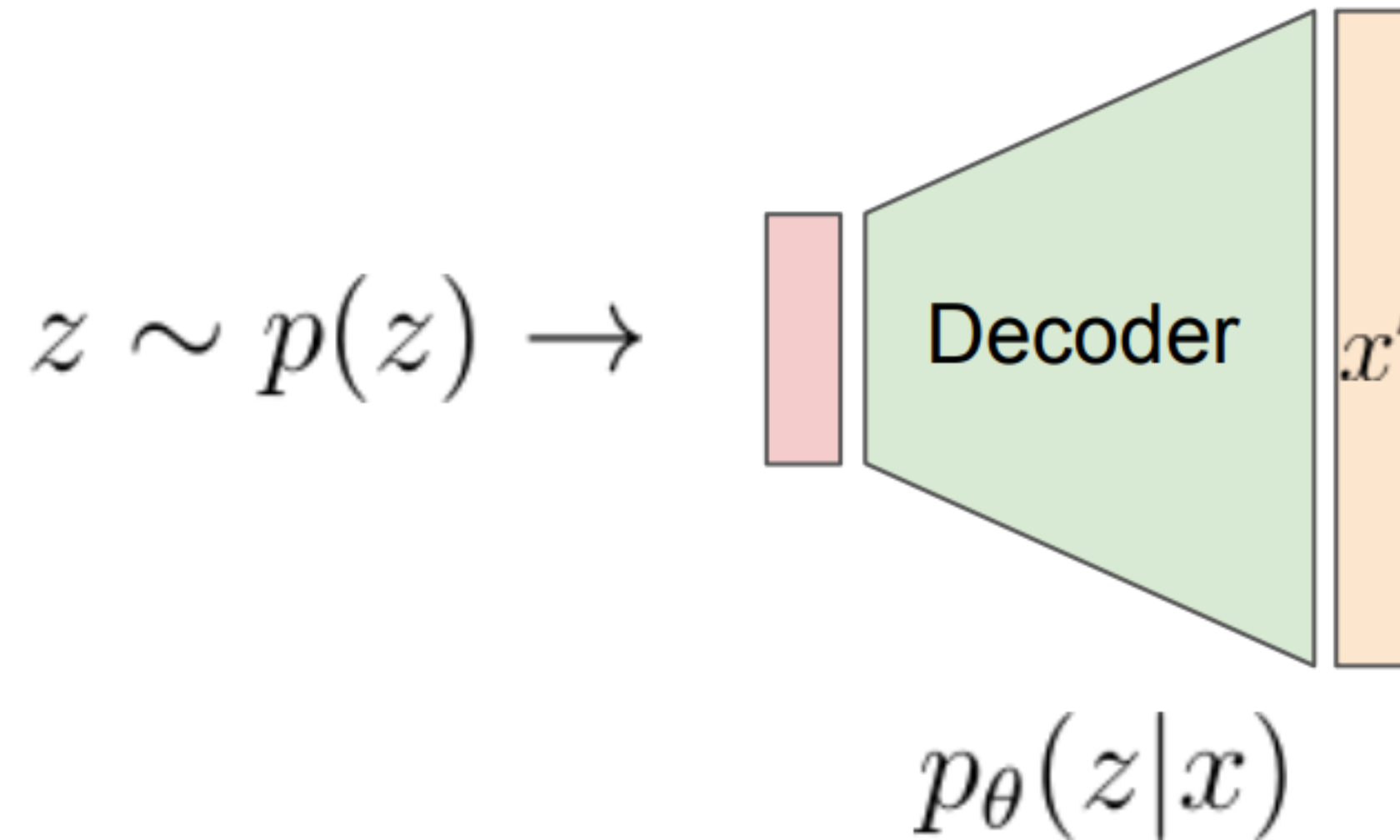


Reconstruction loss: $\mathcal{L}(x, x') = ||x - x'||^2$

Variational AutoEncoder

Can we impose a prior on the latent representation that allows for inference?

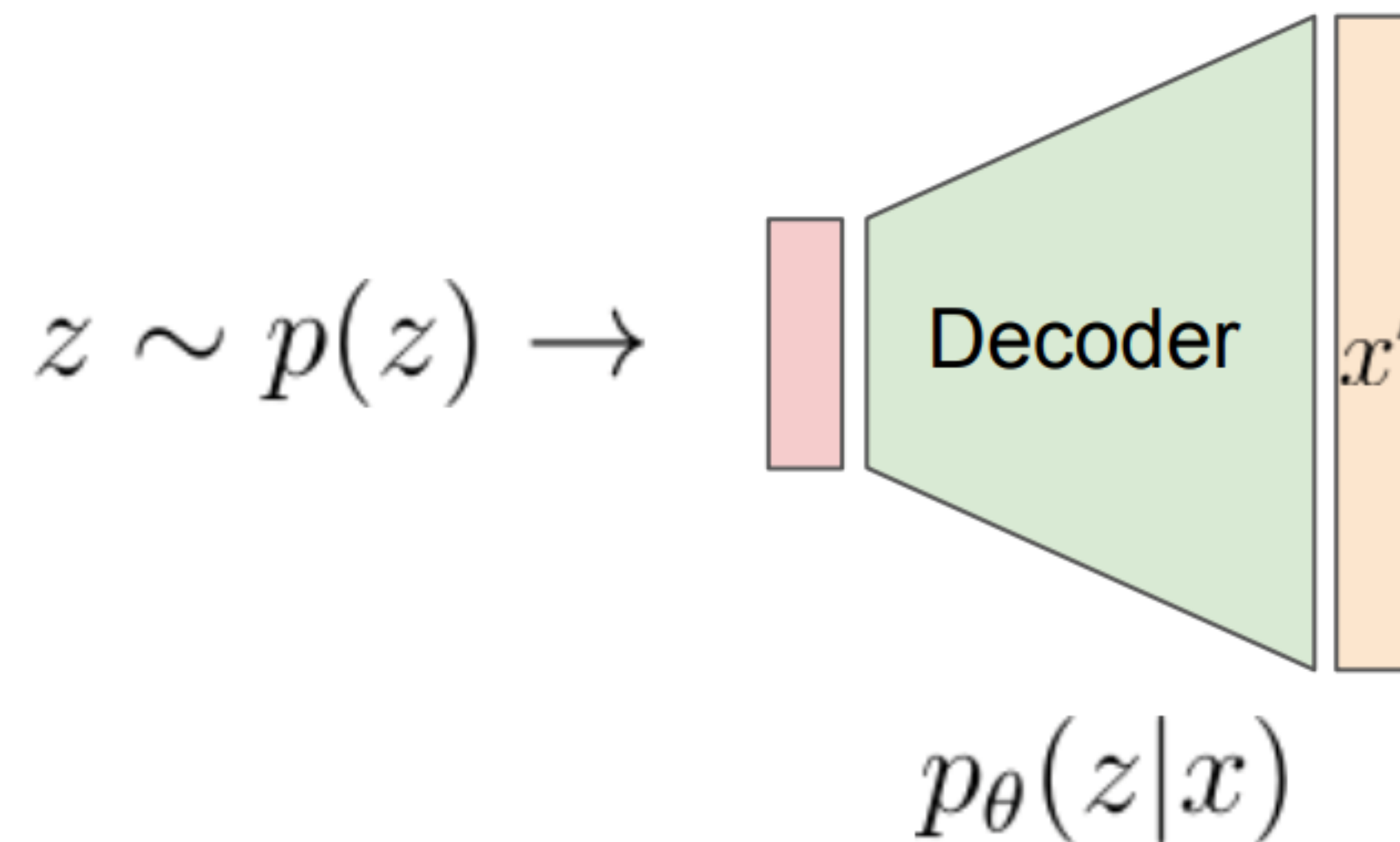
i.e. we want to be able to sample new points from the latent space $z \sim p(z)$ and sample the model posterior distribution $p_{\theta}(z|x)$



Variational AutoEncoder

Can we impose a prior on the latent representation that allows for inference?

i.e. we want to be able to sample new points from the latent space $z \sim p(z)$ and sample the model posterior distribution $p_{\theta}(z|x)$

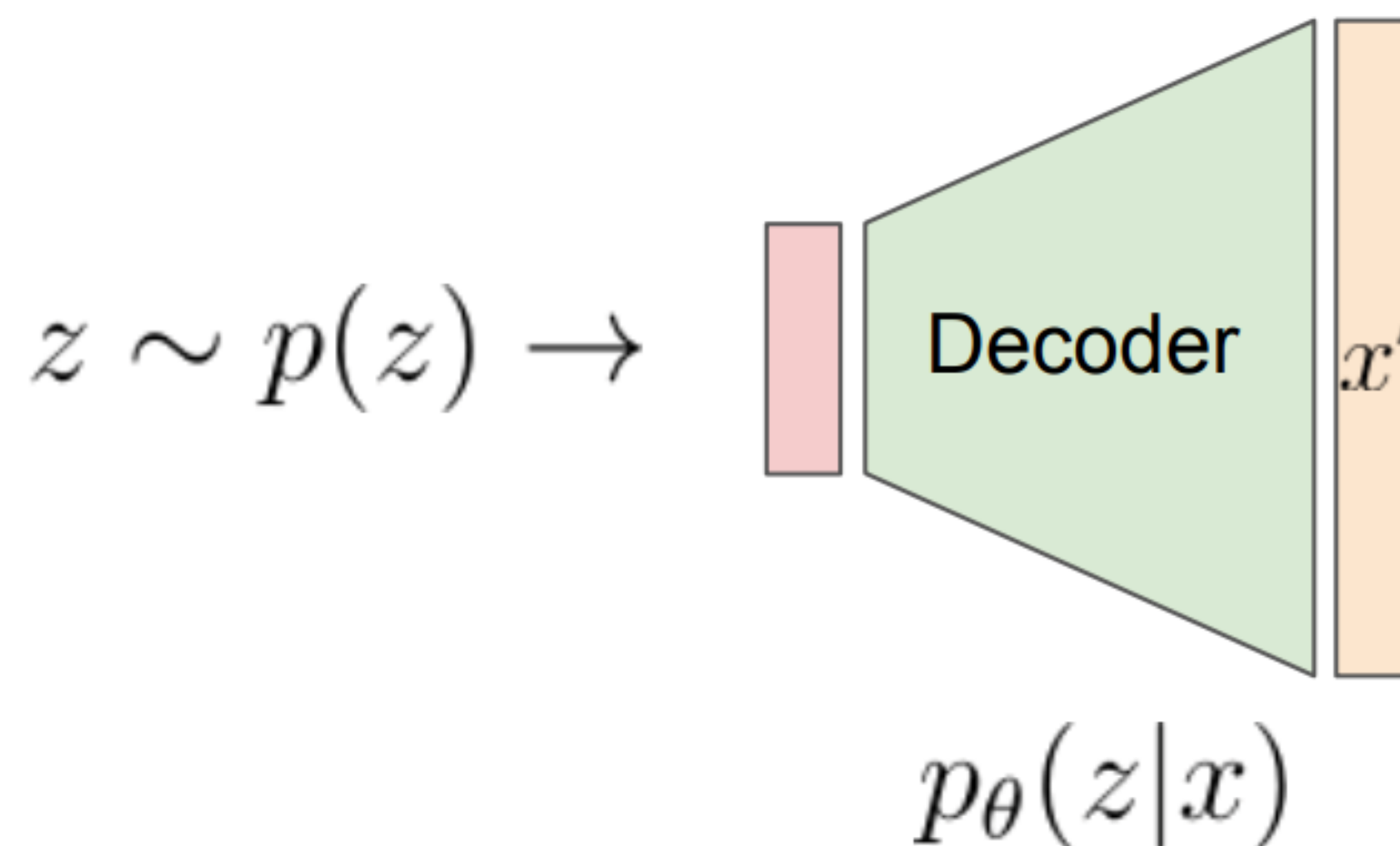


$$p_{\theta}(z|x) = \frac{p_{\theta}(x|z)p(z)}{p_{\theta}(x)}$$

Variational AutoEncoder

Can we impose a prior on the latent representation that allows for inference?

i.e. we want to be able to sample new points from the latent space $z \sim p(z)$ and sample the model posterior distribution $p_{\theta}(z|x)$



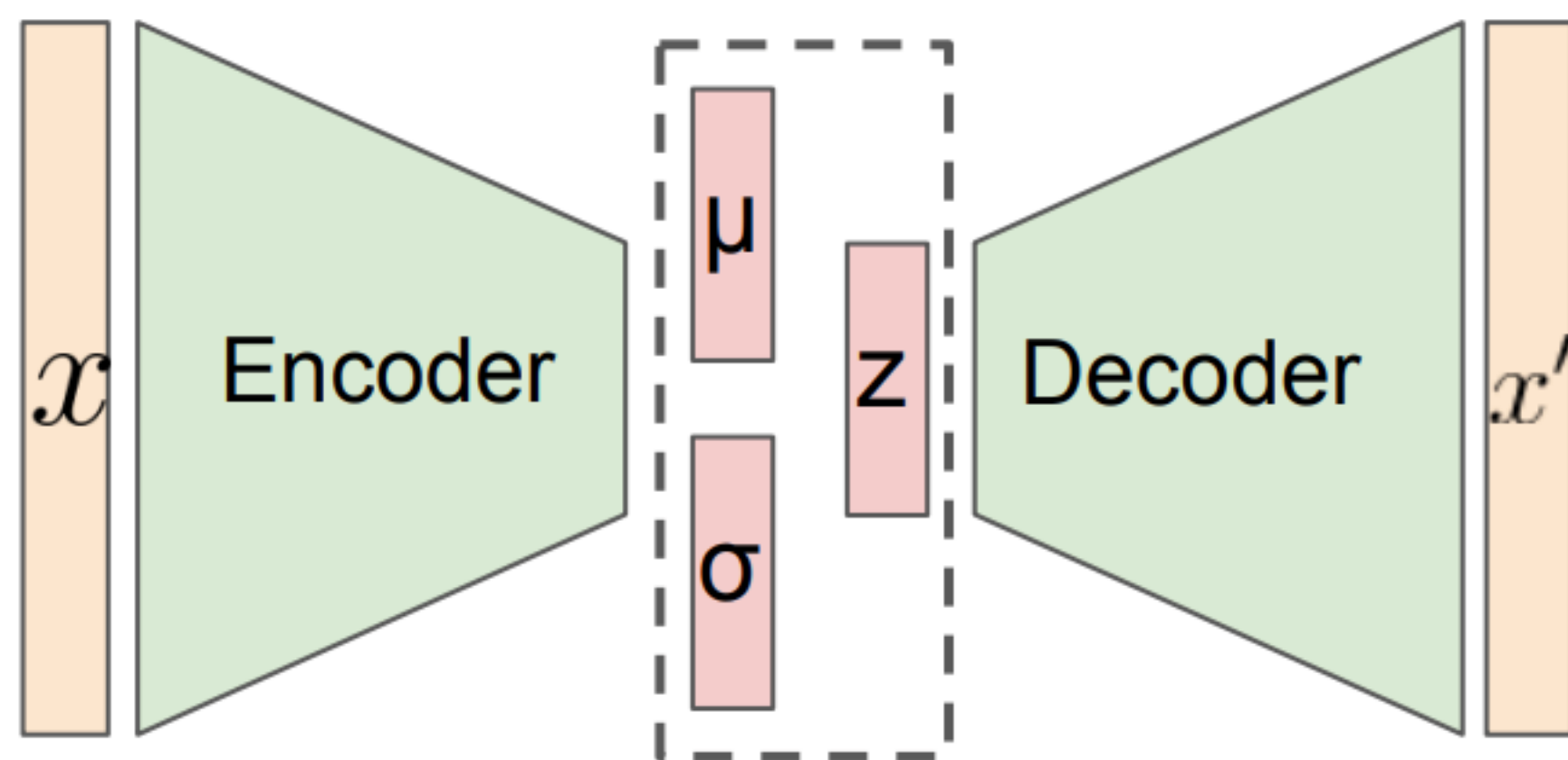
$$p_{\theta}(z|x) = \frac{p_{\theta}(x|z)p(z)}{p_{\theta}(x)}$$

$$q_{\phi}(z|x) \approx p_{\theta}(z|x)$$

Variational AutoEncoder

Add a loss regularization term that “encourages” the encoder to distribution to match a prior.

$$q_{\phi}(z|x) \approx \mathcal{N}(\mu, \sigma)$$



$$\text{VAE loss} = (\text{reconstruction loss}) + D(q_{\phi}(z|x) || p(z))$$

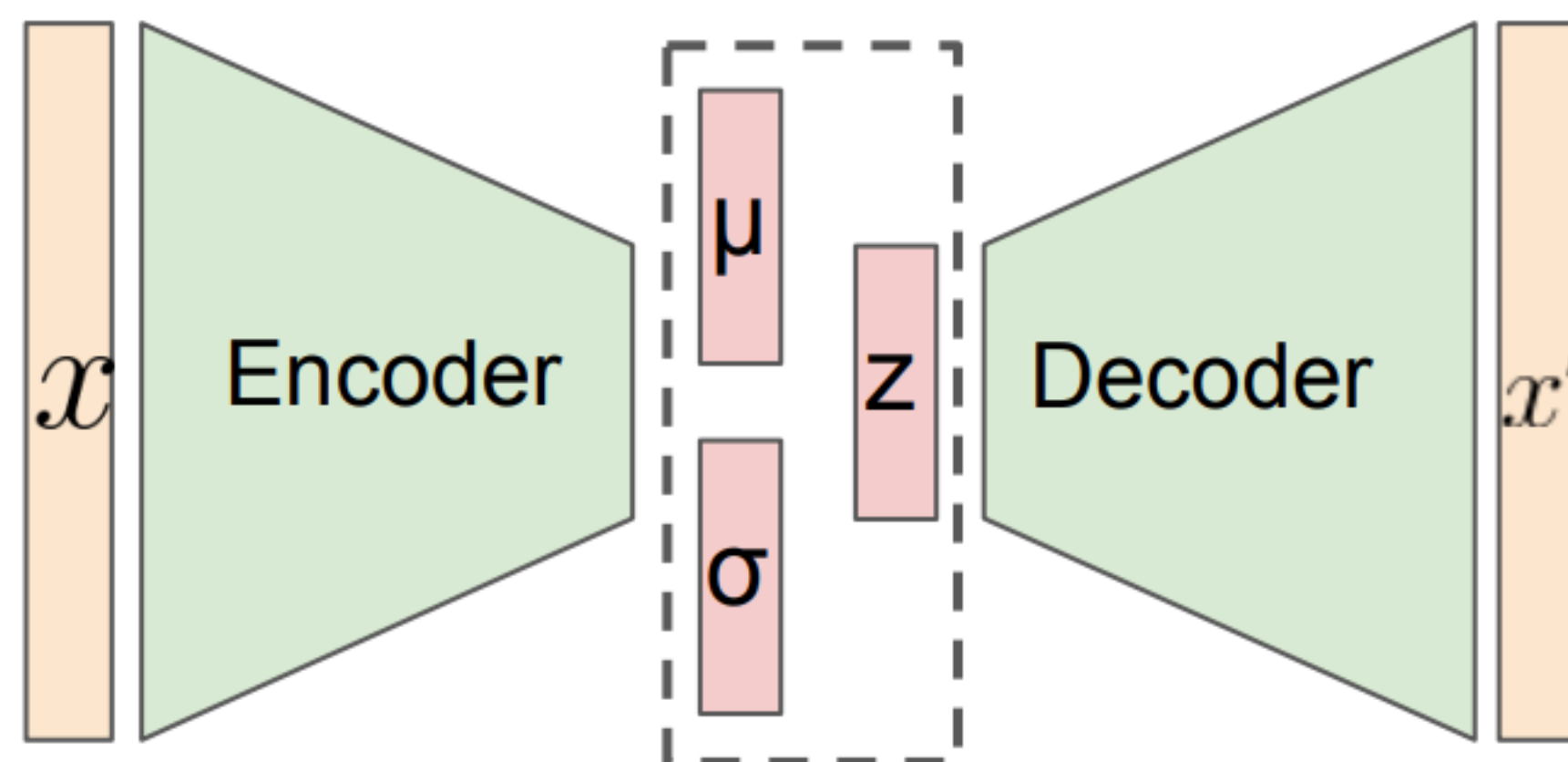
↓

$$D(\mathcal{N}(\mu, \sigma) || \mathcal{N}(0, 1))$$

Variational AutoEncoder

You can't backprop through a stochastic layer → use the reparameterization trick

$$q_{\phi}(z|x) \approx \mathcal{N}(\mu, \sigma) \rightarrow \sigma * \epsilon + \mu \quad \text{where } \epsilon \sim \mathcal{N}(0, 1)$$

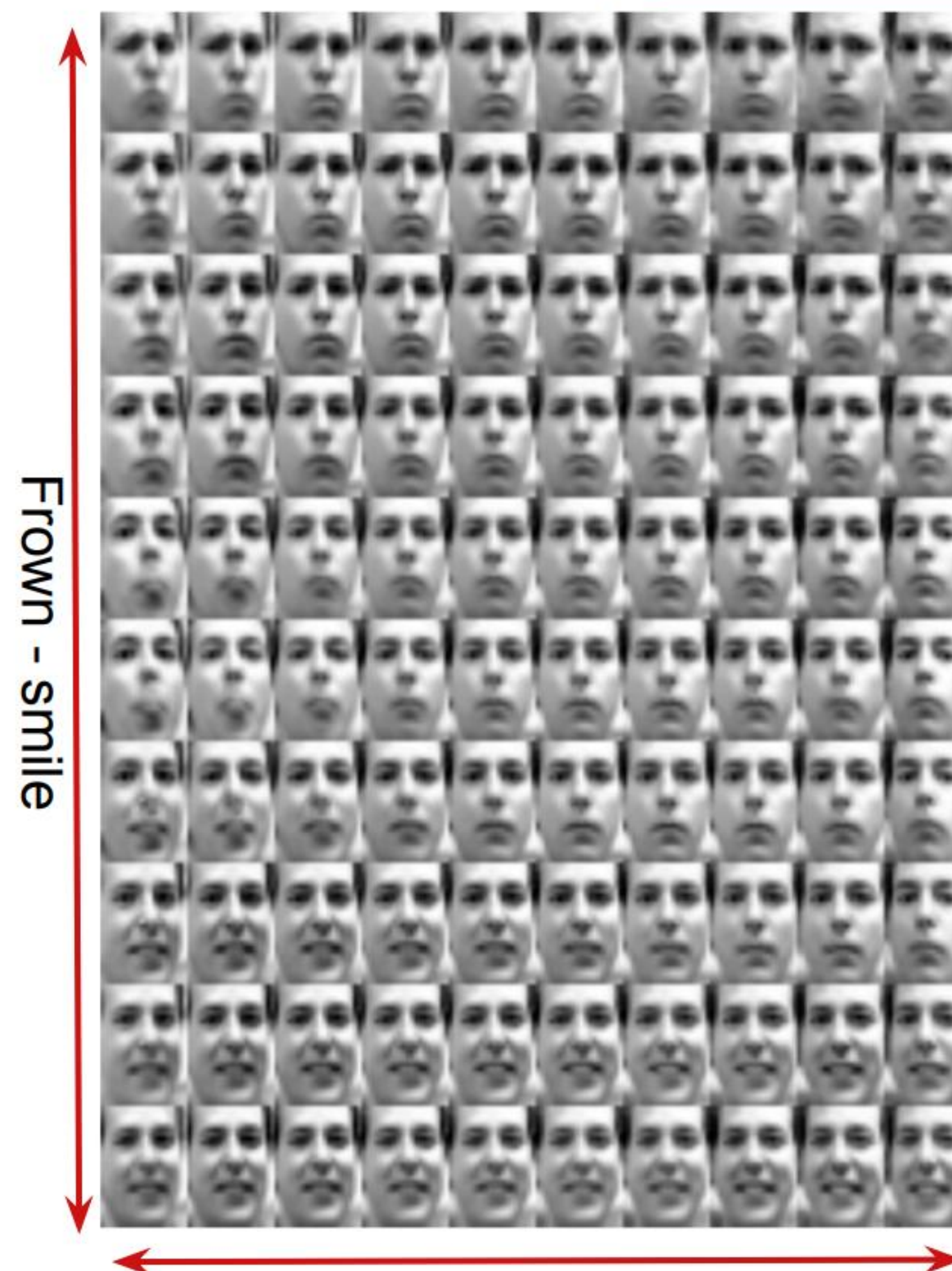
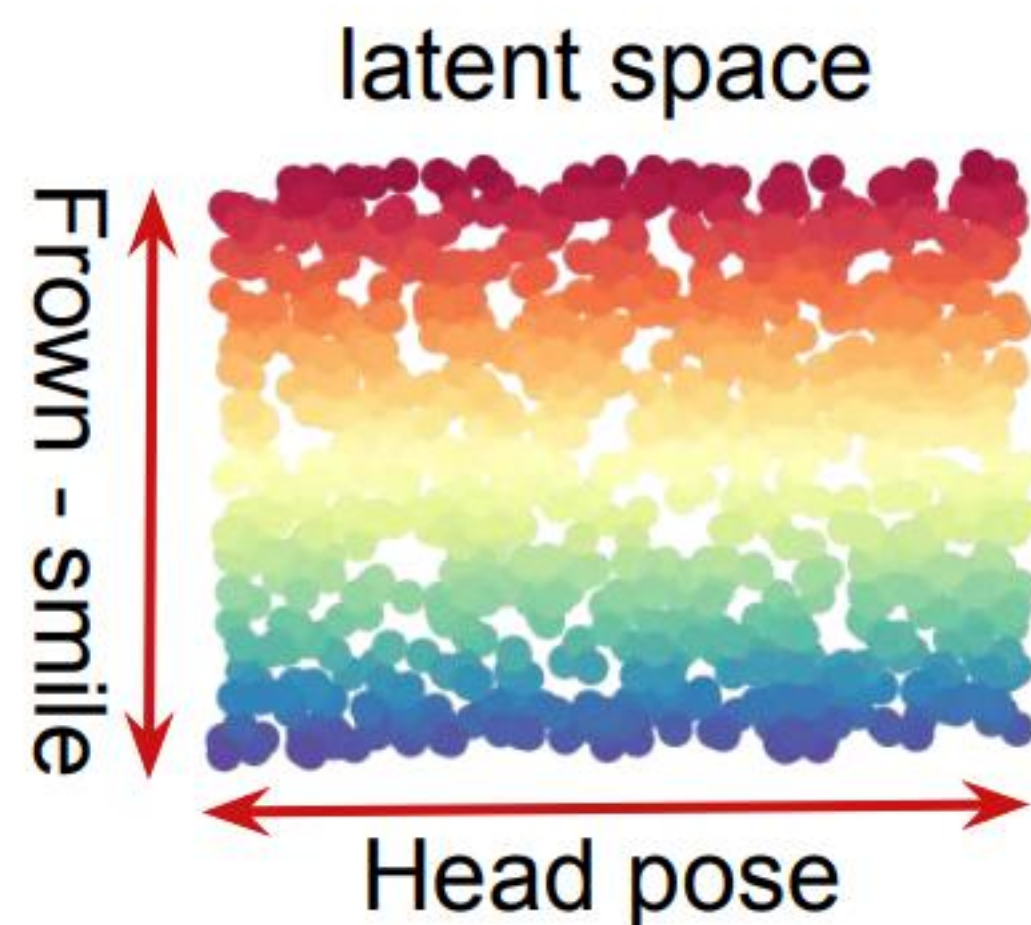


$$\text{VAE loss} = (\text{reconstruction loss}) + D(q_{\phi}(z|x) || p(z))$$

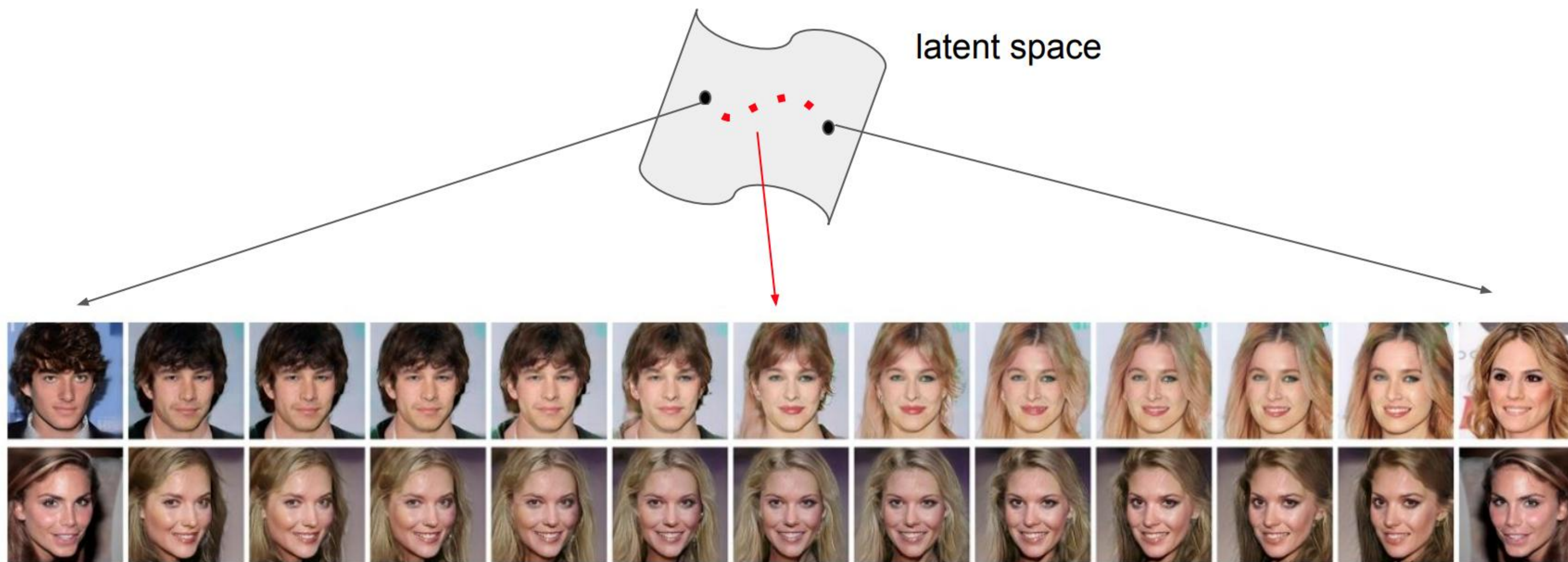
$$\downarrow$$
$$D(\mathcal{N}(\mu, \sigma) || \mathcal{N}(0, 1))$$

Semantically meaningful directions in the learned latent space

Kingma & Welling, [arXiv:1312.6114](https://arxiv.org/abs/1312.6114)



Interpolations in the latent space correspond to smooth transitions over the generated data manifold

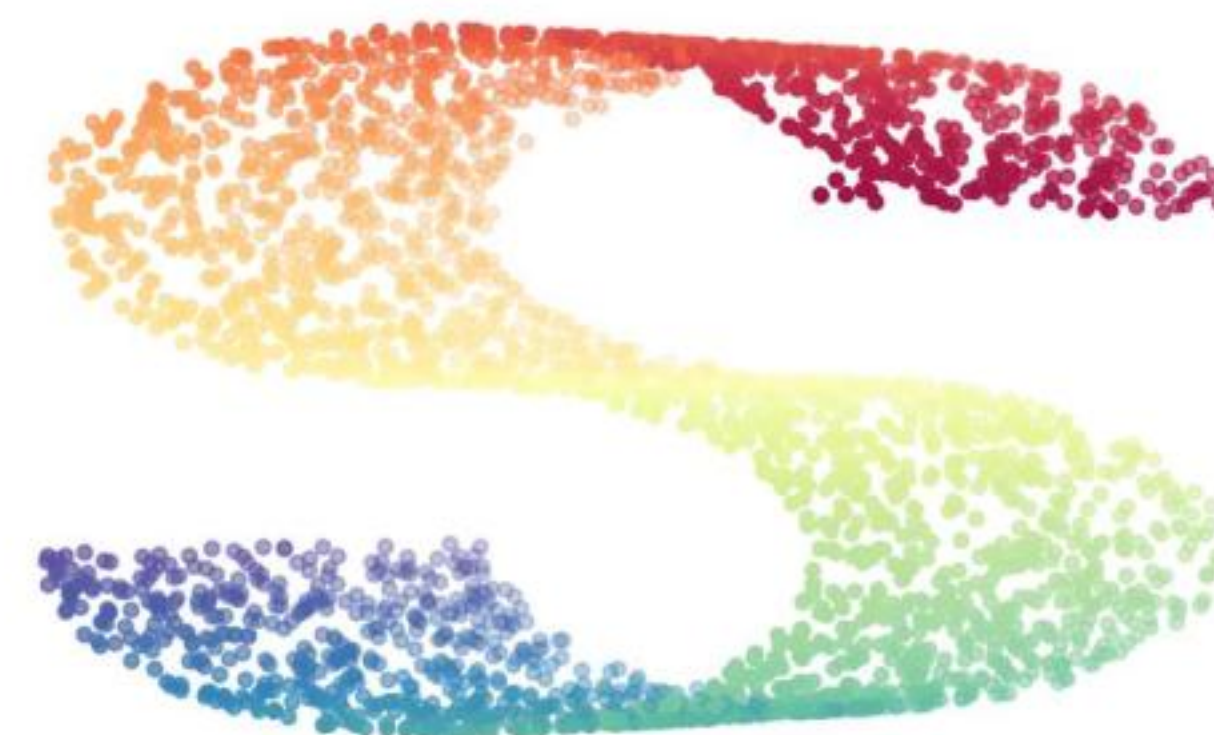
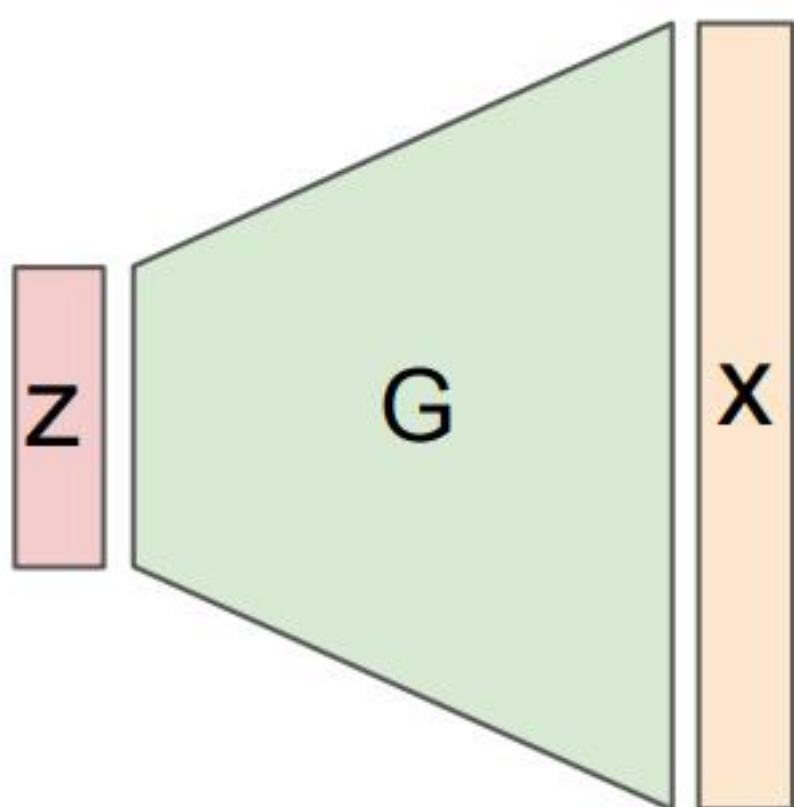


"A MAN" "A WOMAN" "A MAN" "A WOMAN" "A MAN" "A WOMAN" "A MAN" "A WOMAN" "A MAN" "A WOMAN" "A MAN" "A WOMAN"

Generative Adversarial Networks

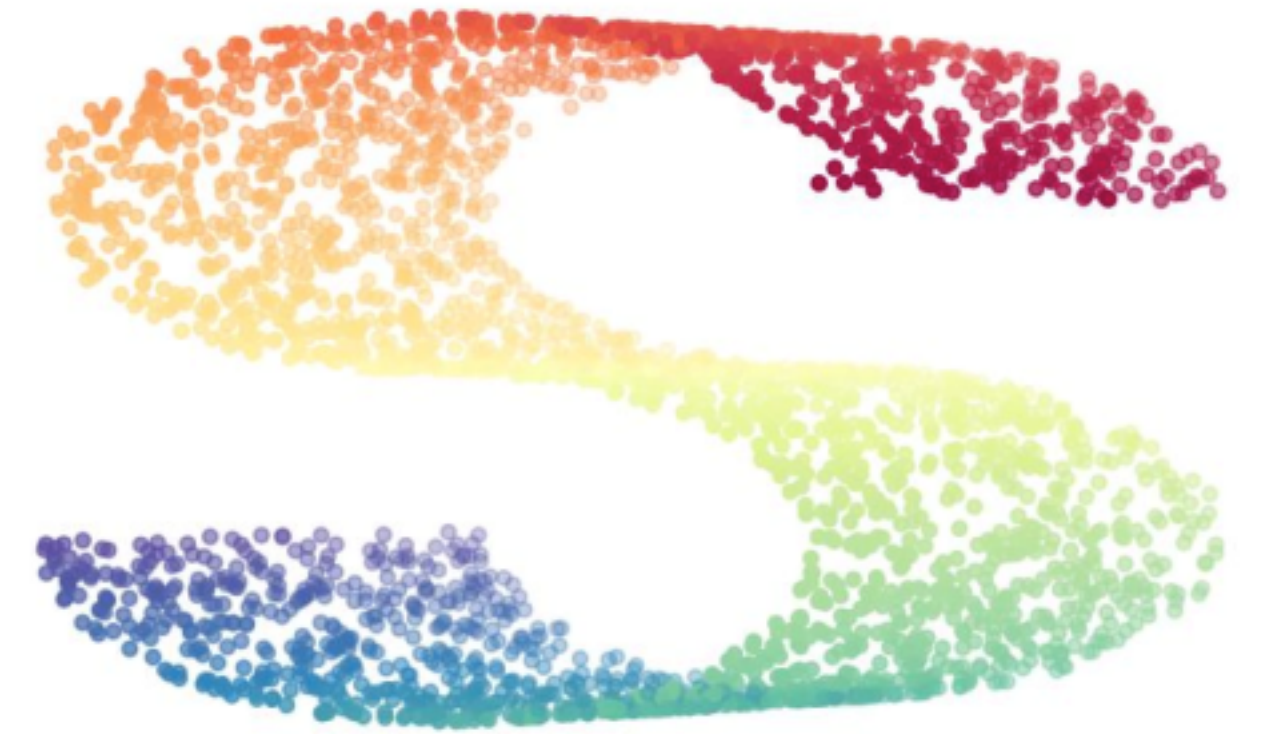
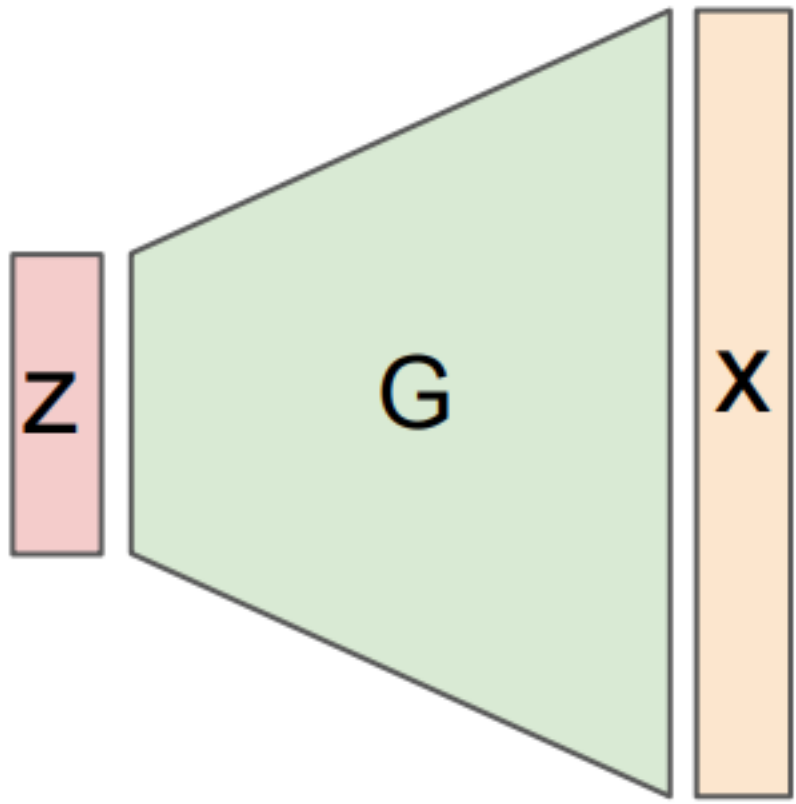
Generative Adversarial Networks (GANs)

We want to learn how to generate samples that look similar to the real-data; approximately from the real data manifold.



Generative Adversarial Networks (GANs)

We want to learn how to generate samples that look similar to the real-data; approximately from the real data manifold.

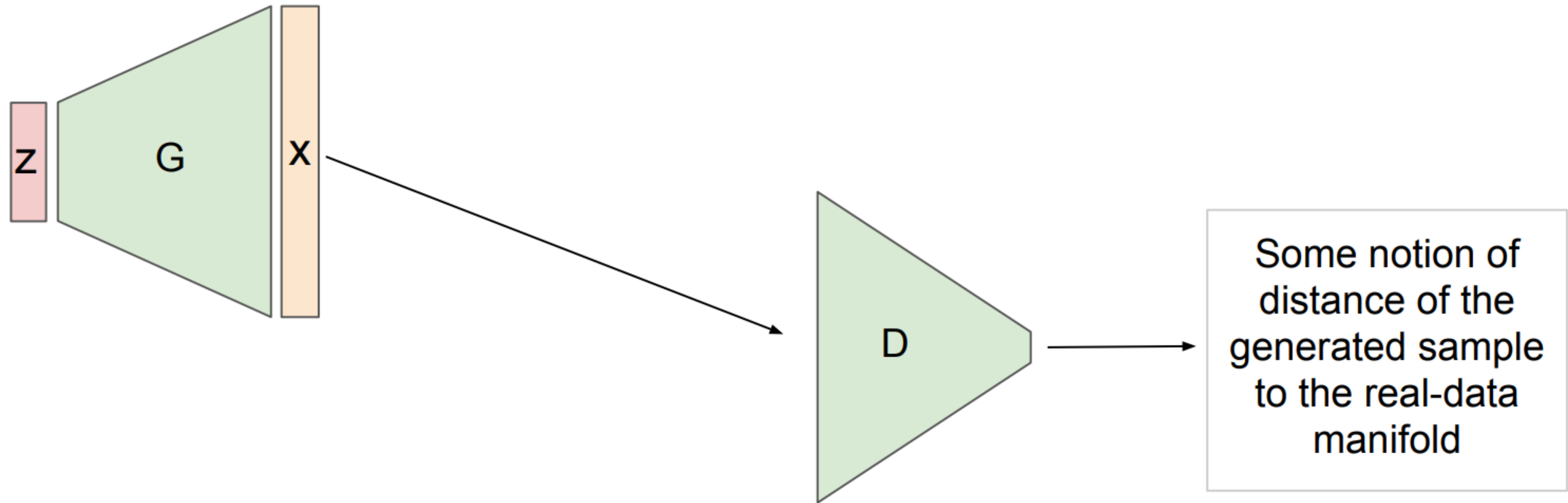


In other words, we need a loss function that tells us how close the generated data is to the real-data manifold:

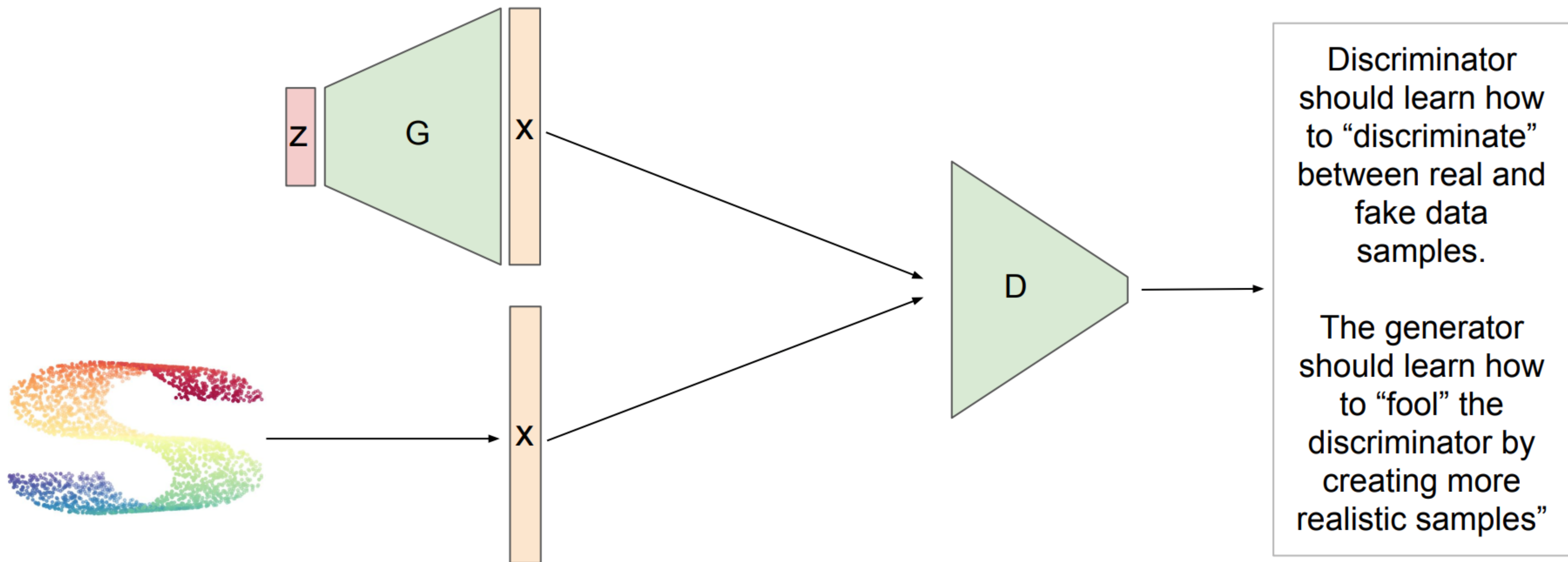
$$\mathcal{L} = f(G(z))$$

Can we learn such an f ?

Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)



The new network (discriminator/critic) is trained “adversarially.”

GANs' loss (original loss)

The discriminators is trained to minimize:

$$J^{(D)} = -\frac{1}{2}\mathbb{E}_{x \sim \mathbb{P}_{data}} \log D(x) - \frac{1}{2}\mathbb{E}_{z \sim p_z} \log(1 - D(G(z)))$$



learn to give a high score
to samples from realdata



learn to give a low score
to samples from
generated data

GANs' loss (original loss)

The discriminators is trained to minimize:

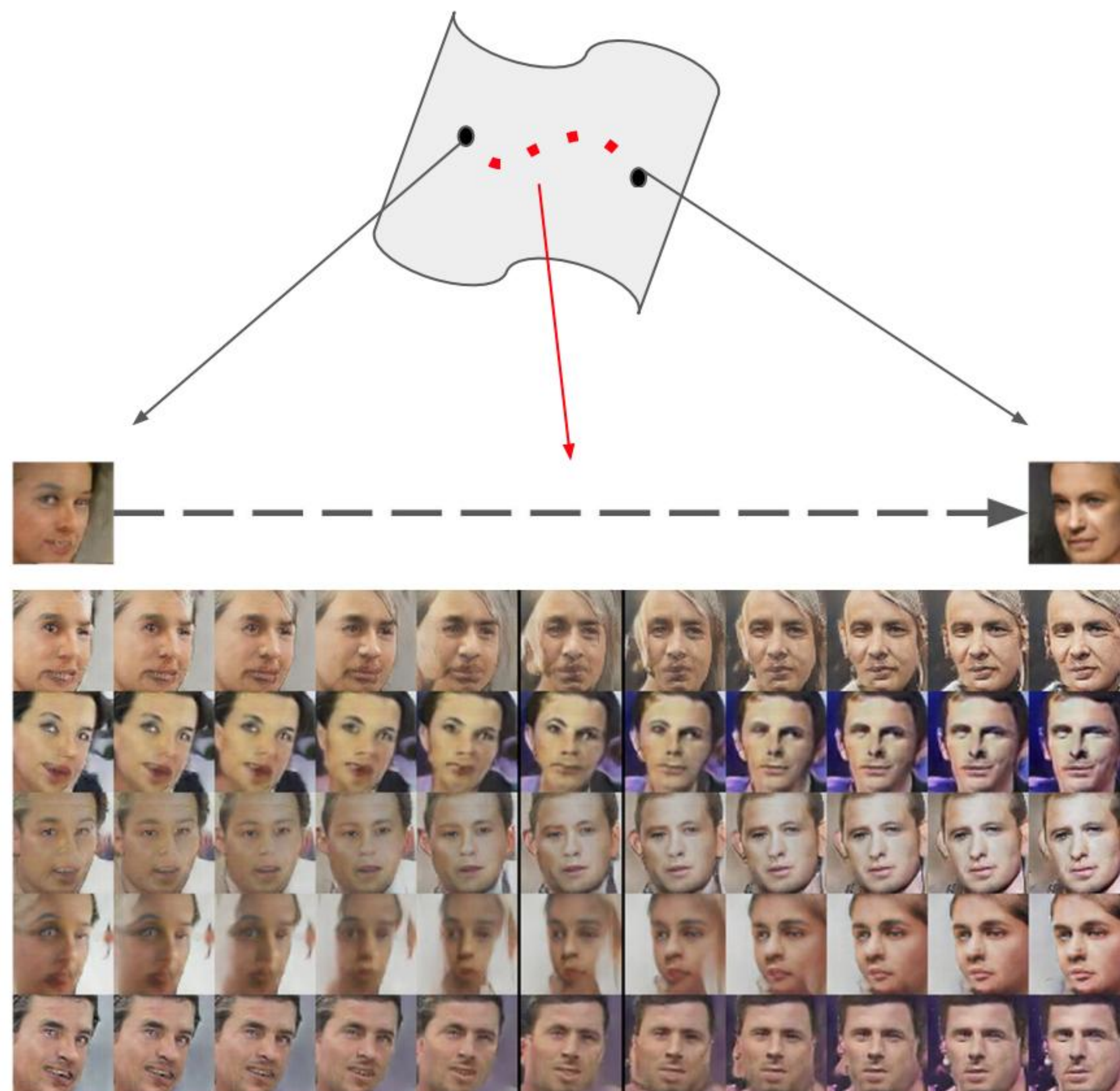
$$J^{(D)} = -\frac{1}{2}\mathbb{E}_{x \sim \mathbb{P}_{data}} \log D(x) - \frac{1}{2}\mathbb{E}_{z \sim p_z} \log(1 - D(G(z)))$$

In the minimax game formulation, you train the generator to maximize the discriminators loss (to fool the discriminator):

$$J^{(G)} = -J^{(D)}$$

$$J^{(G)} = -\frac{1}{2}\mathbb{E}_{z \sim p_z} \log D(G(z))$$

Exploring the learned latent representation: Interpolation



Exploring the learned latent representation: directions in the latent space are semantically meaningful (simple arithmetics)

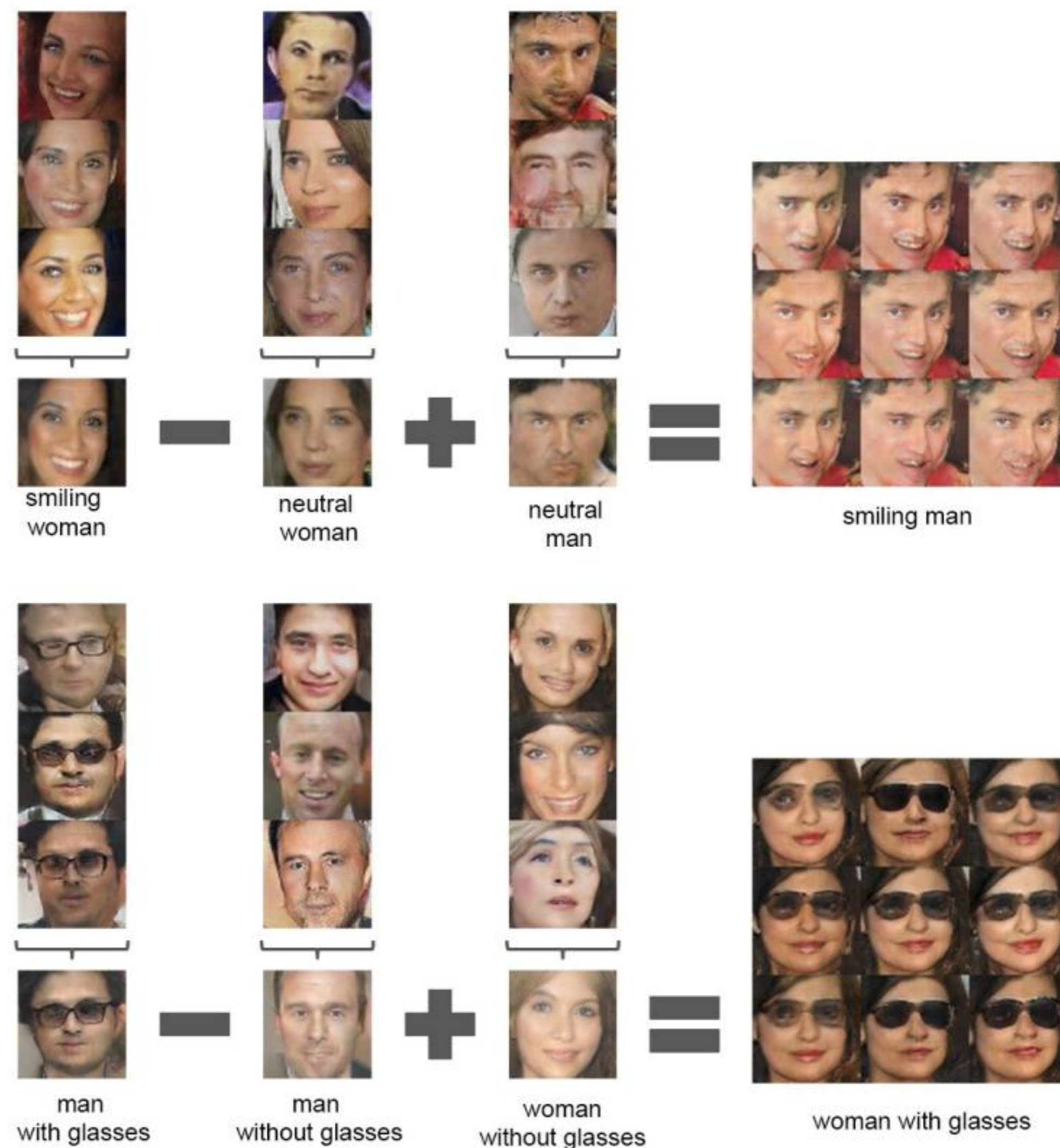
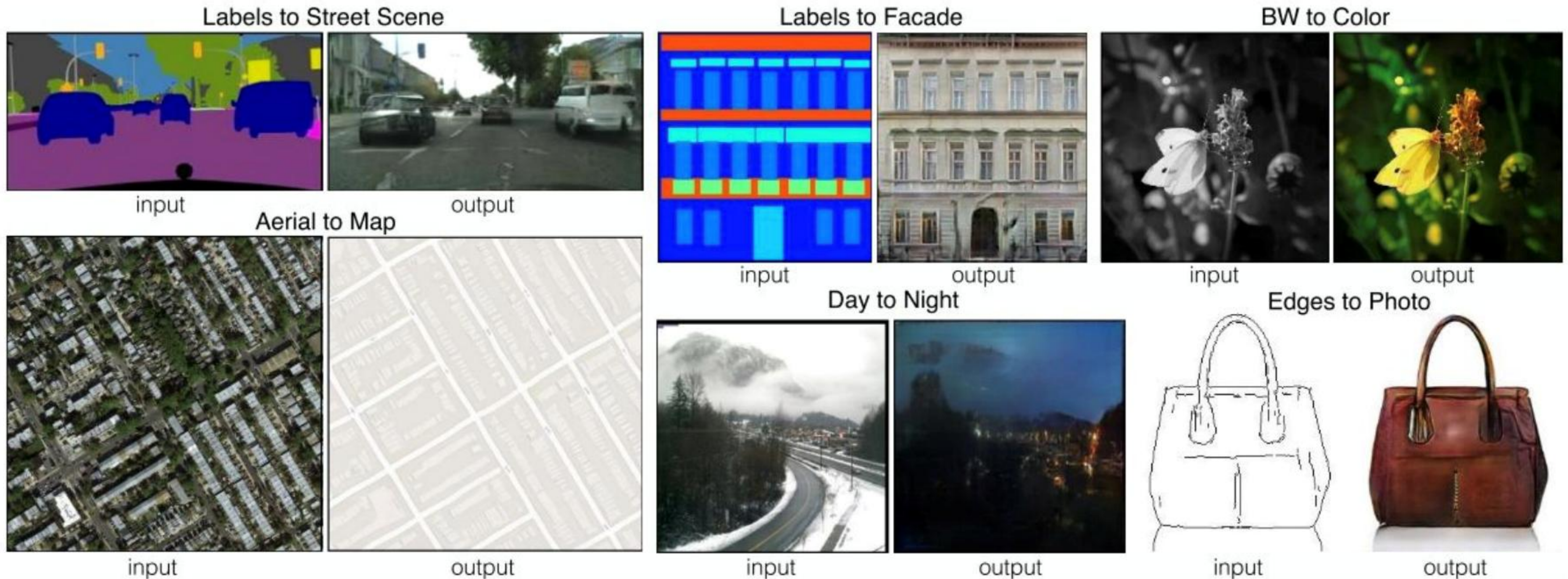


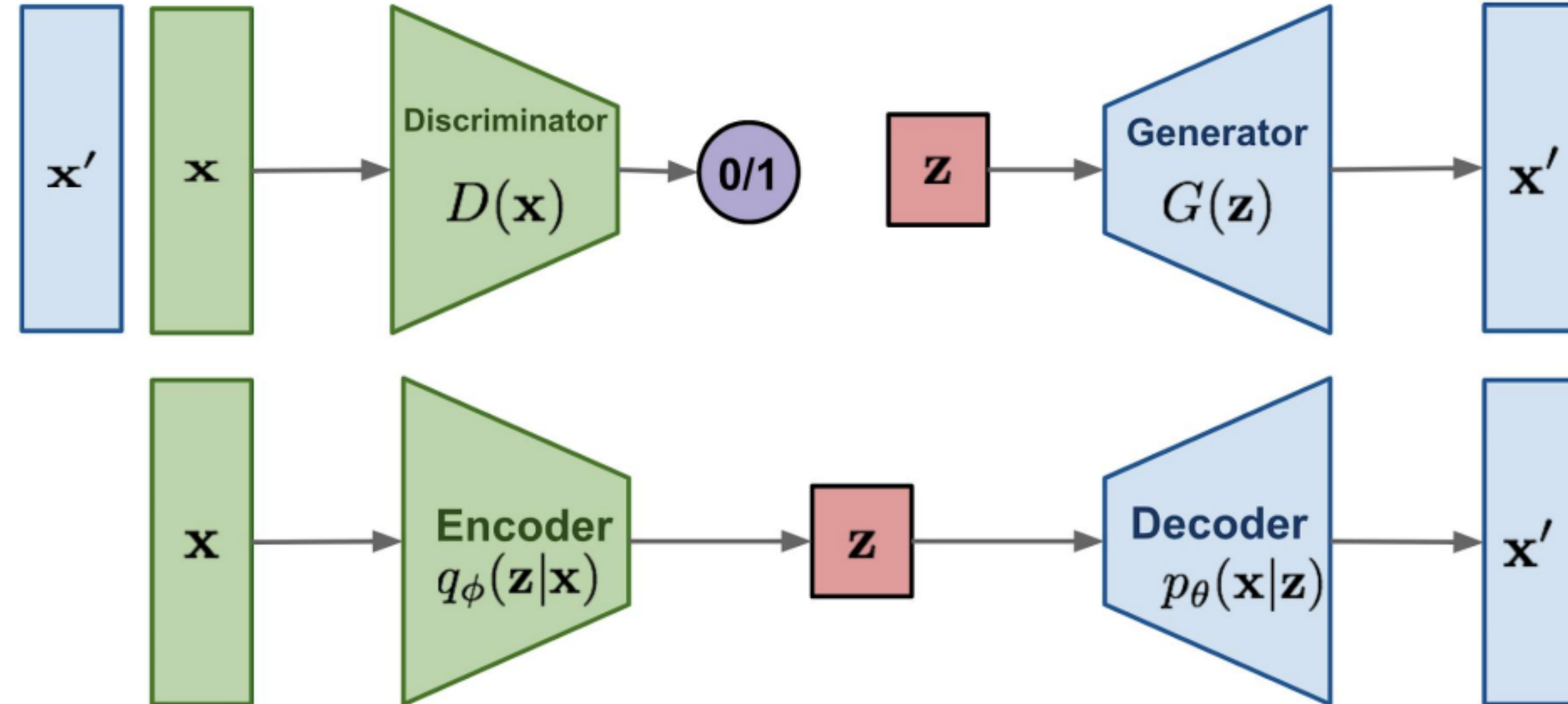
Image to Image translation using conditional GANs

Learns a one-to-one mapping between domain X and Y, training with paired images.



VAEs vs GANs

GAN: minimax the classification error loss.



VAE



GAN



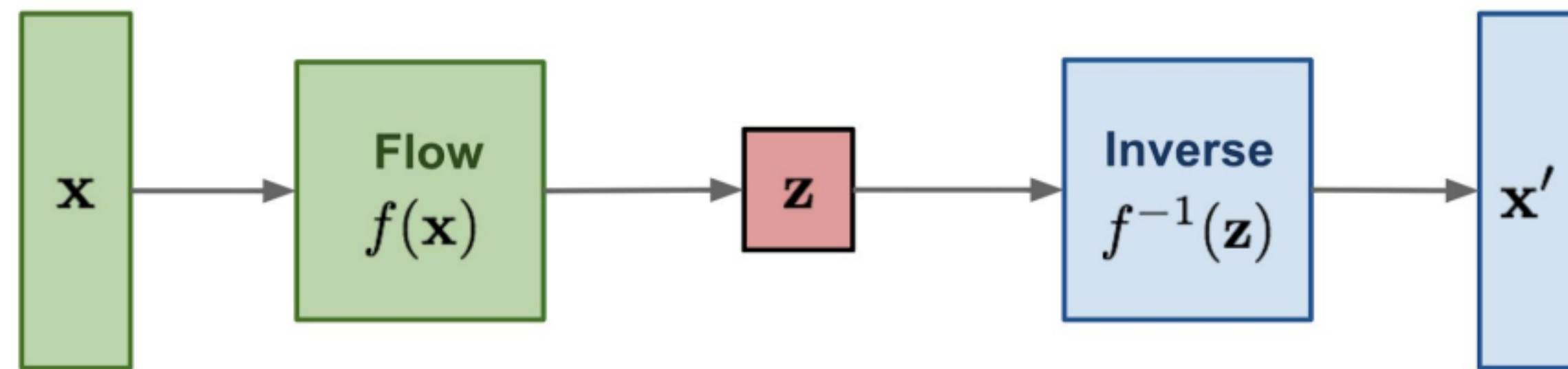
Used mostly for image denoising and anomaly detection and generation.

Used mostly for super resolution, image to image translation, image generation etc.

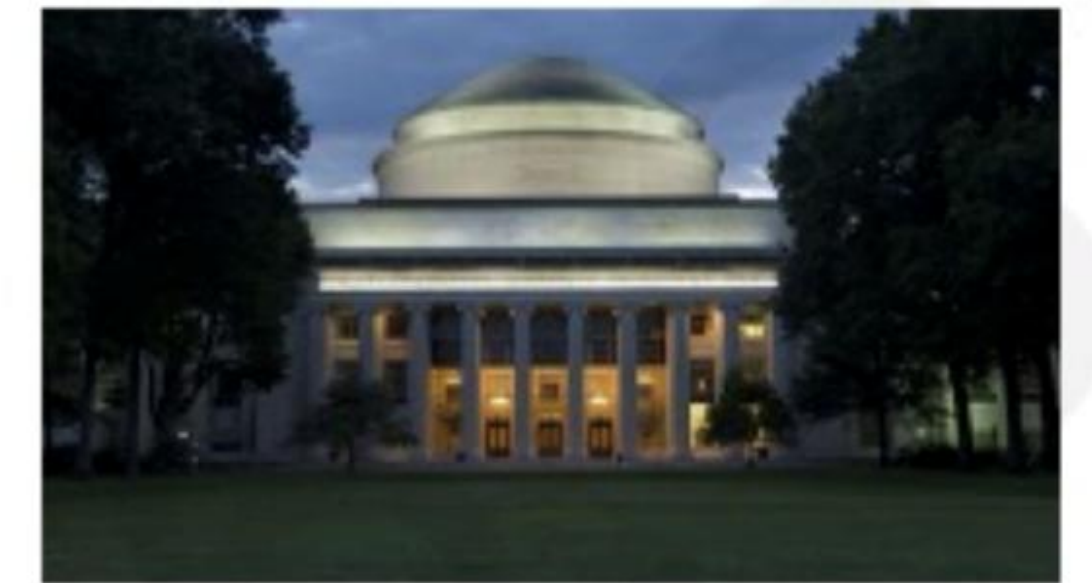
Generative Modelling Landscape

- VAEs and GANS are difficult to train
- Suffer from convergence issues and mode collapse
- Normalising flows and diffusion models

**Flow-based
generative models:**
minimize the negative
log-likelihood

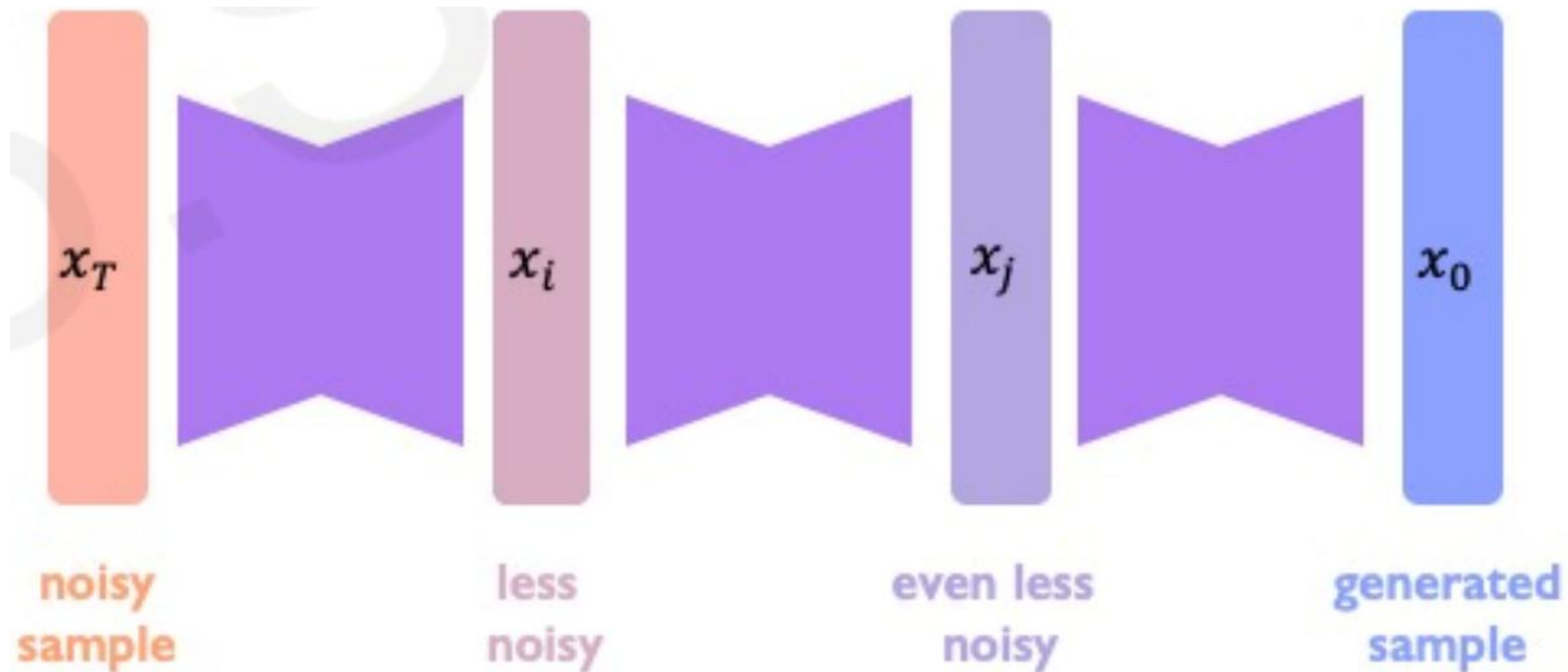


Diffusion Models

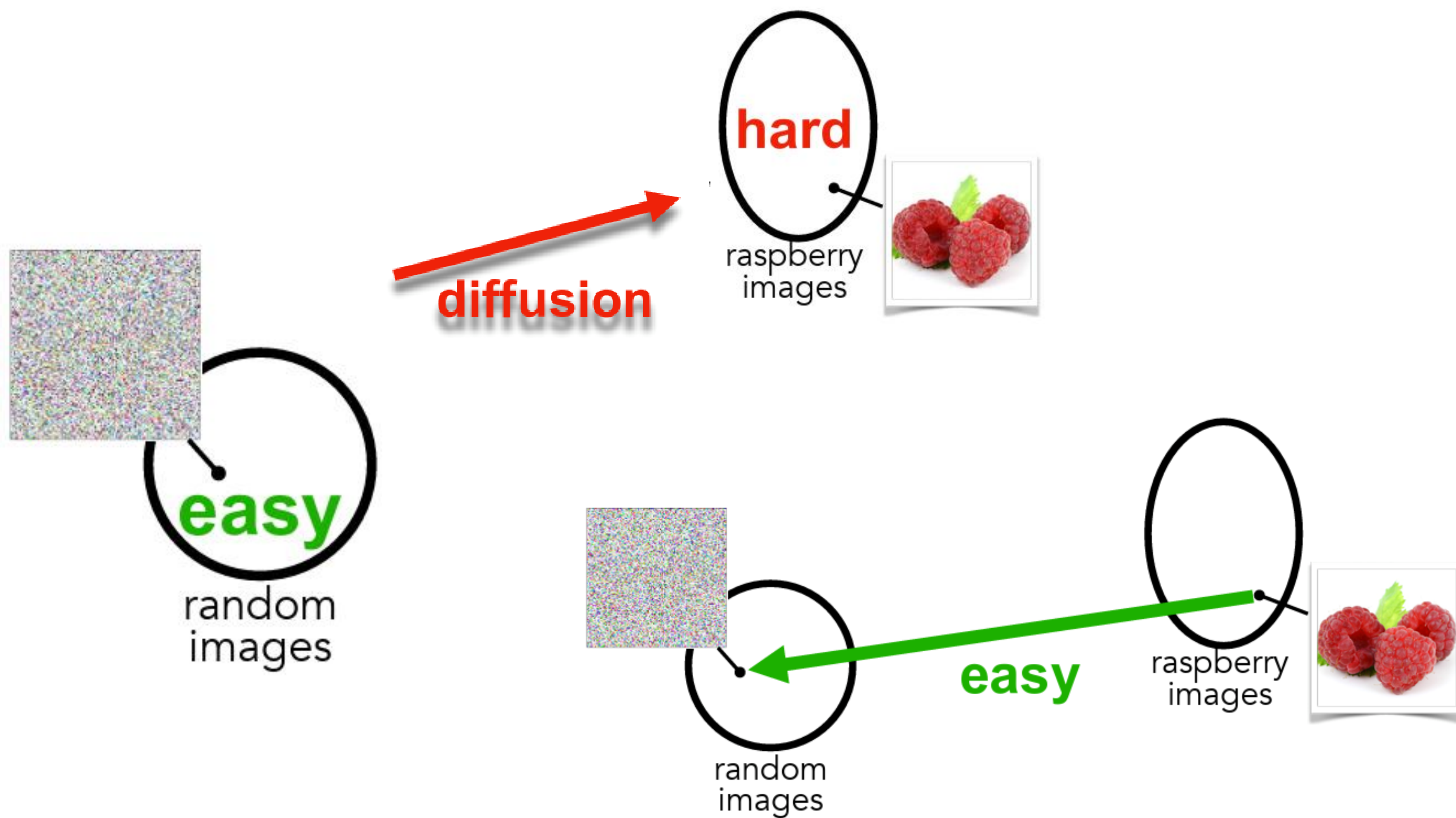


Diffusion models

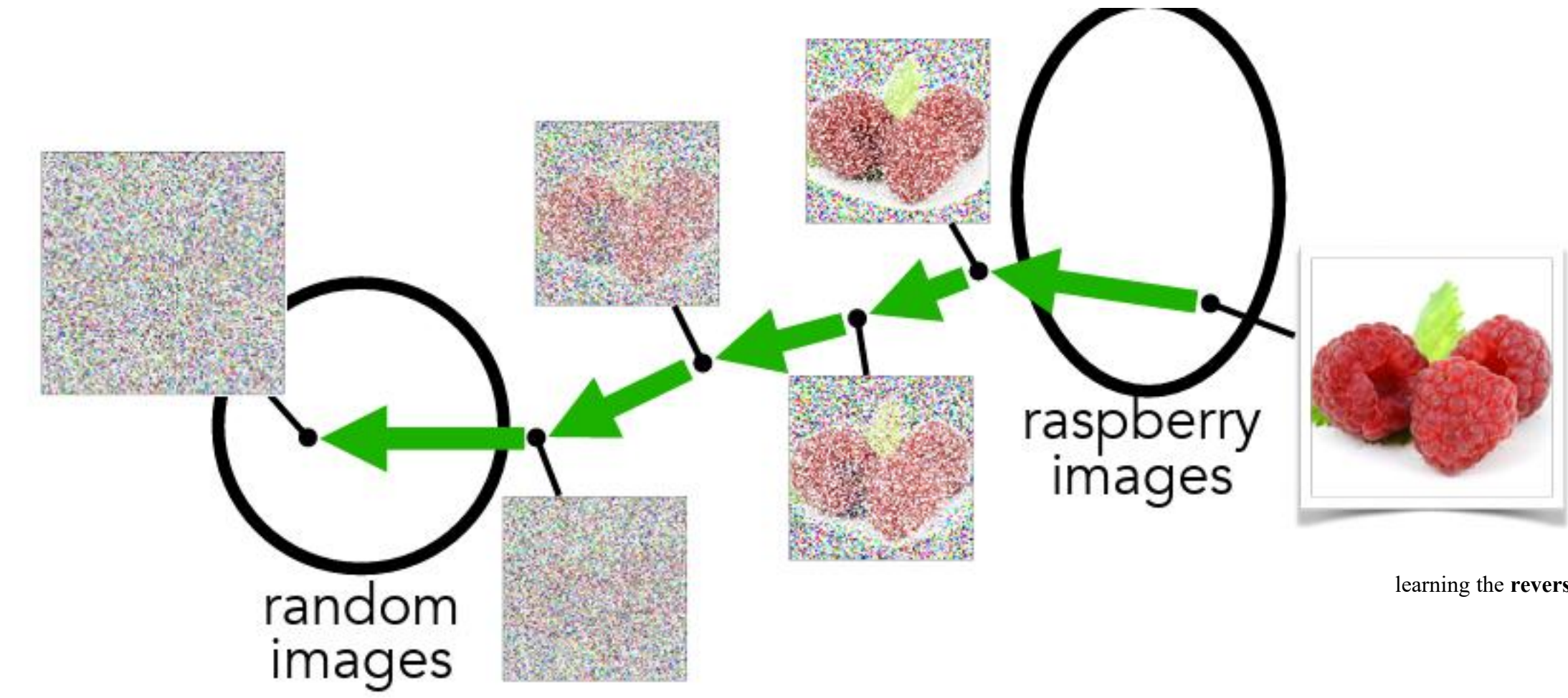
Diffusion: Generating samples iteratively by repeatedly refining and removing noise



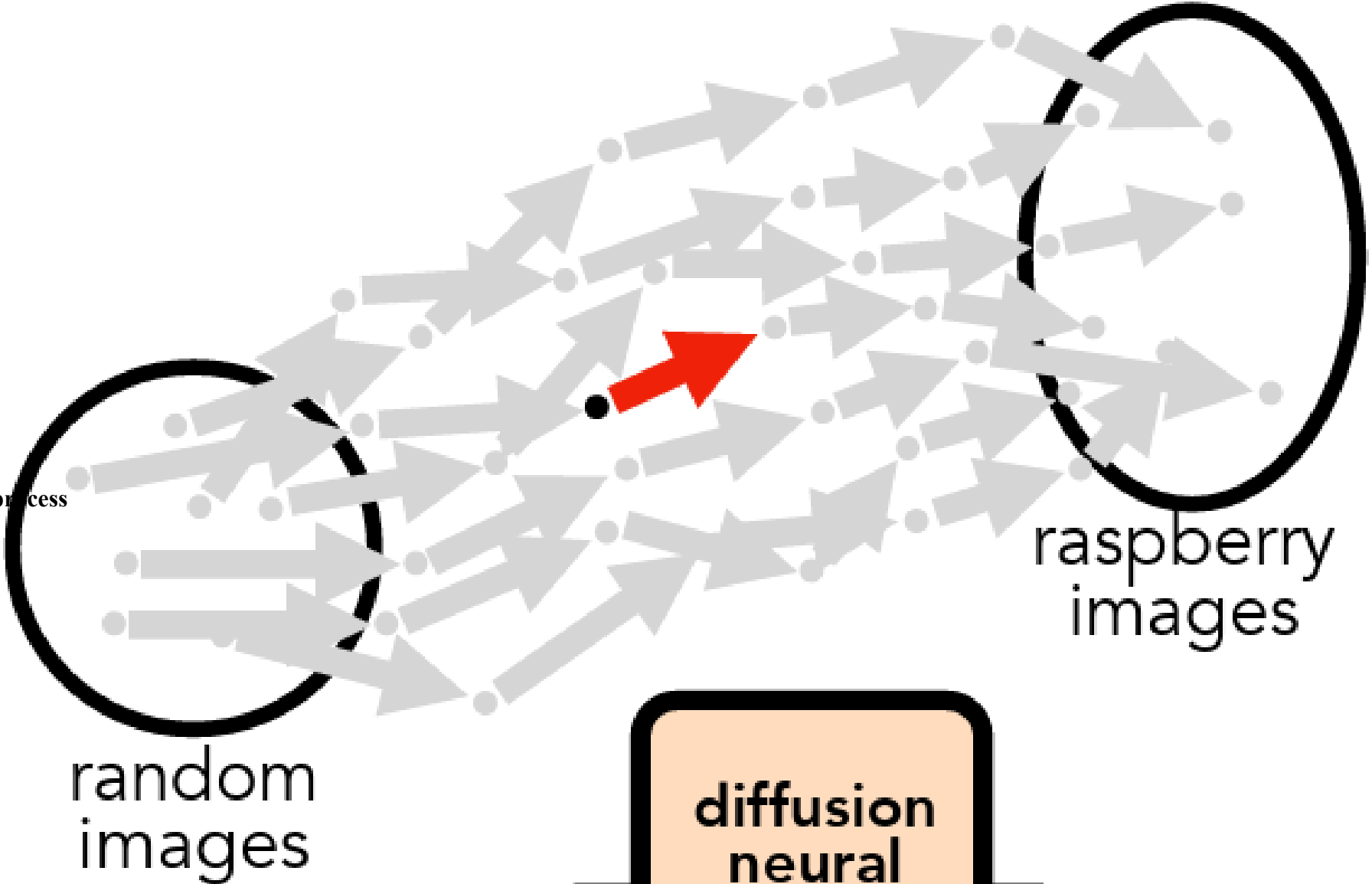
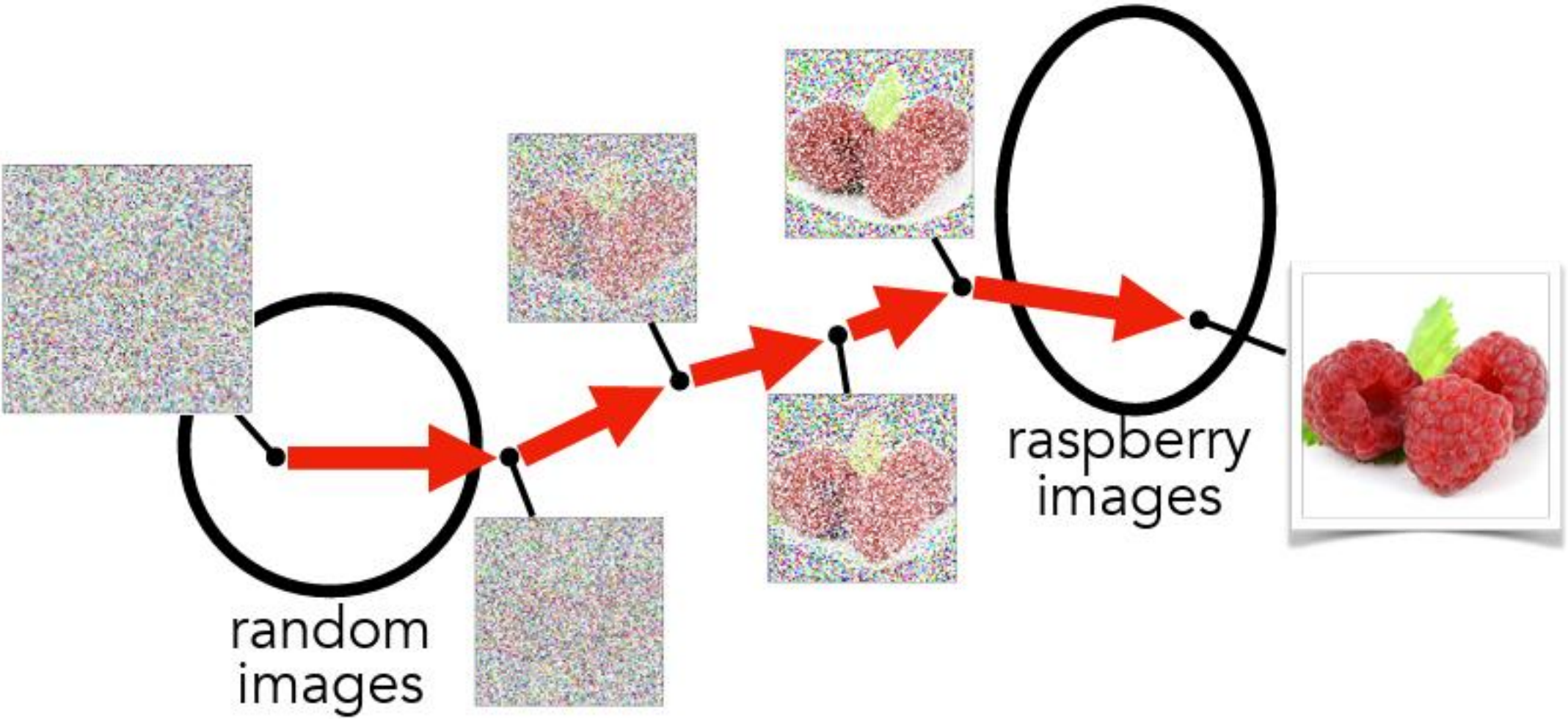
Diffusion



Diffusion



learning the reverse of a noising process



learn the reverse of a noising process

Training

Diffusion (Forward Process)

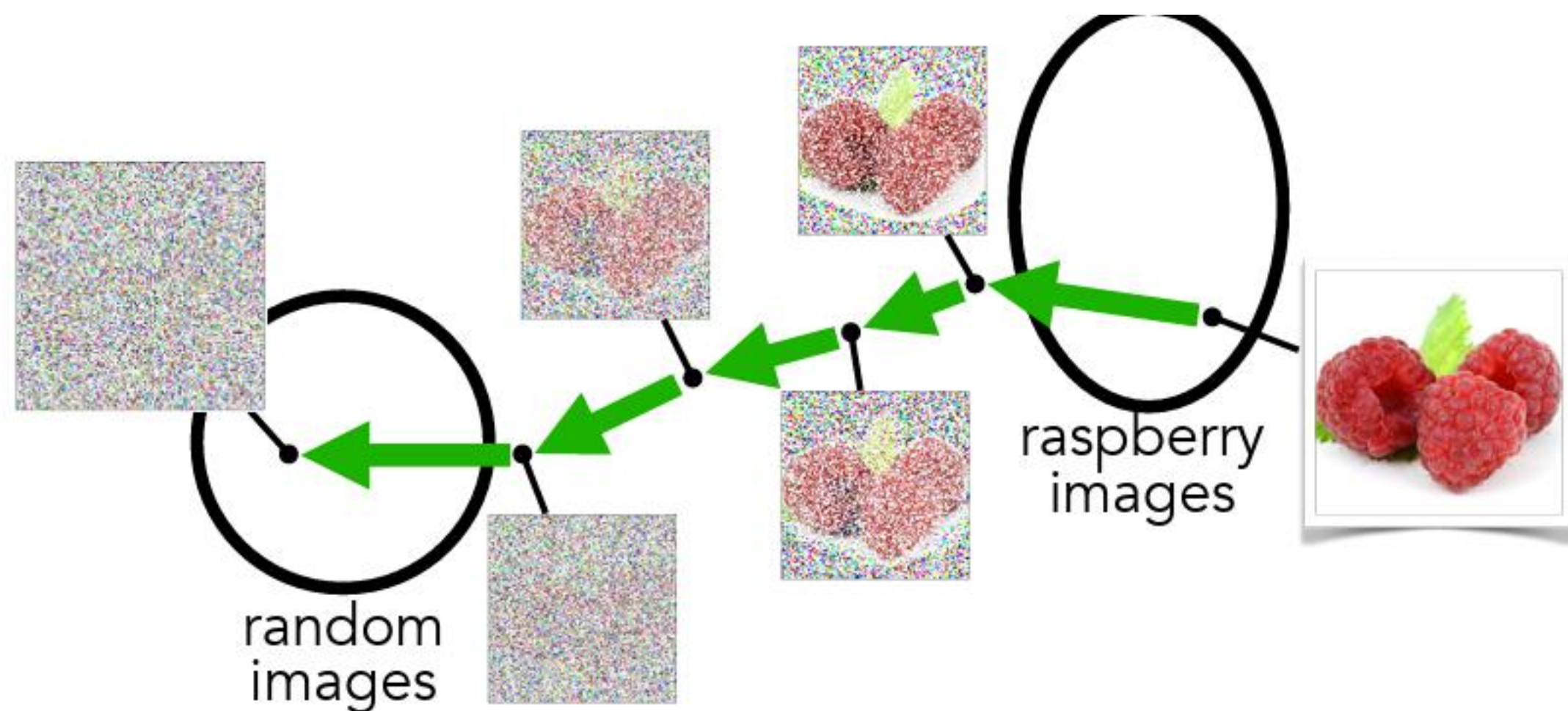
Forward process (known, fixed)

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

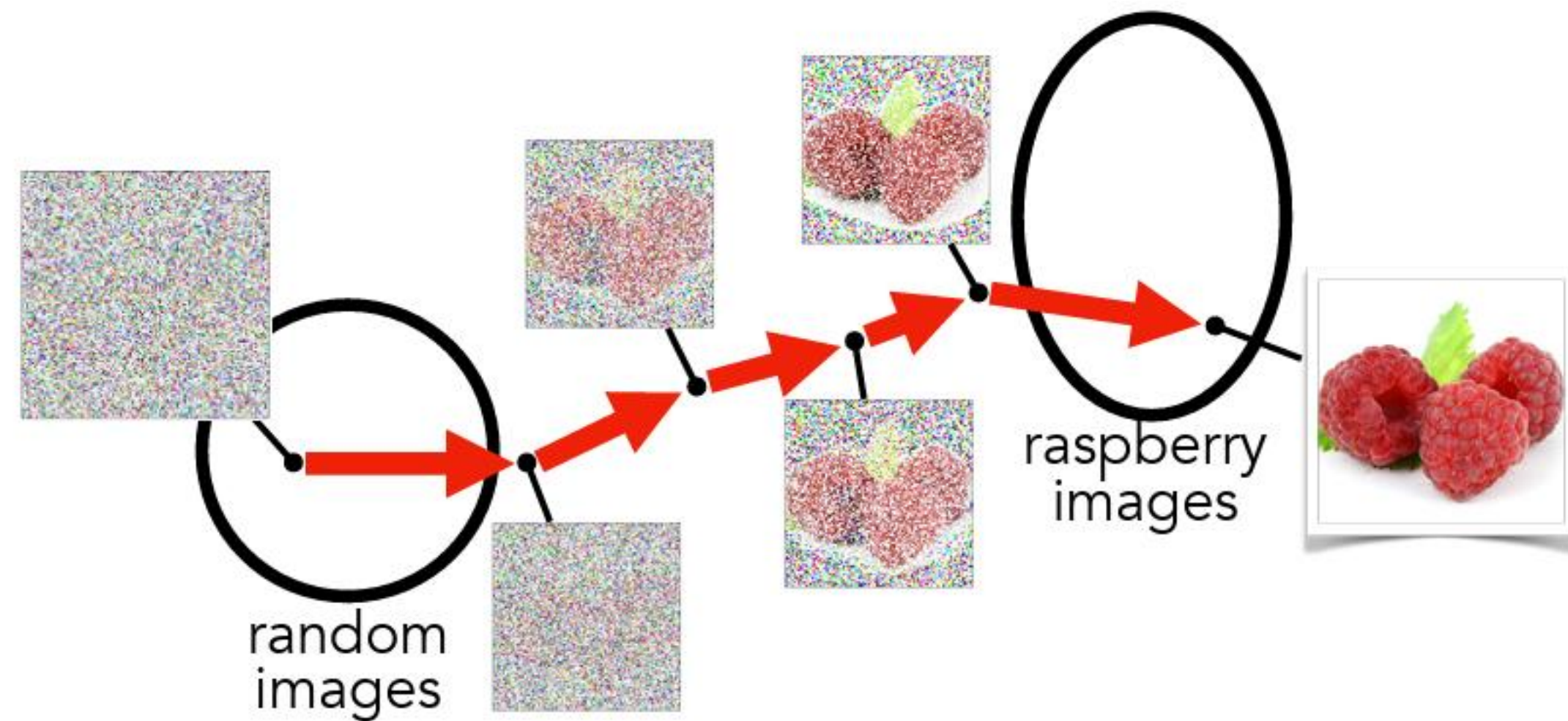
- Slowly destroys structure
- After many steps: $x_T \sim \mathcal{N}(0, I)$

In general at any step t

$$q(x_t|x) = \mathcal{N}(x_t; \sqrt{\alpha_t}x, (1 - \alpha_t)I) \quad \alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau).$$

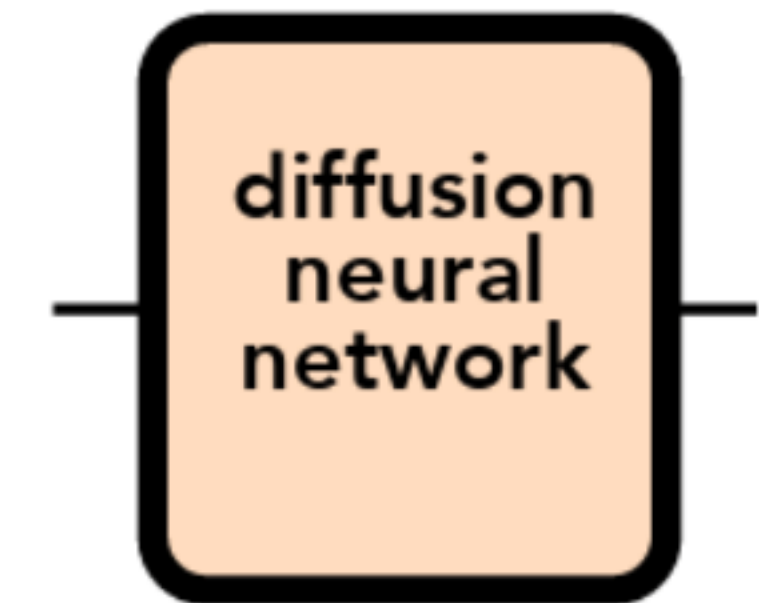


Diffusion : Reverse Process (Generation)



Trains a single neural network to learn

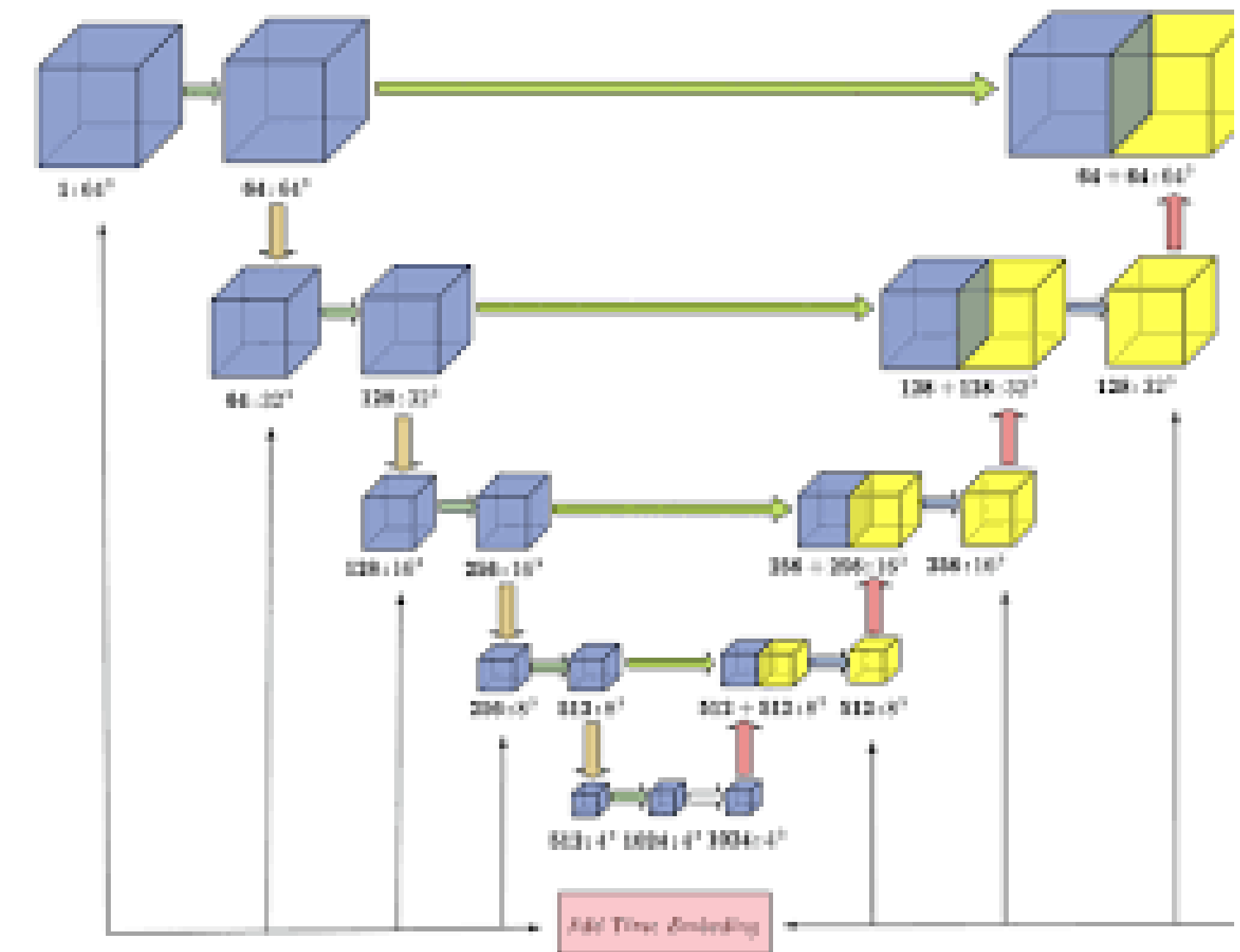
$$\mu(x_t, \mathbf{w}, t)$$



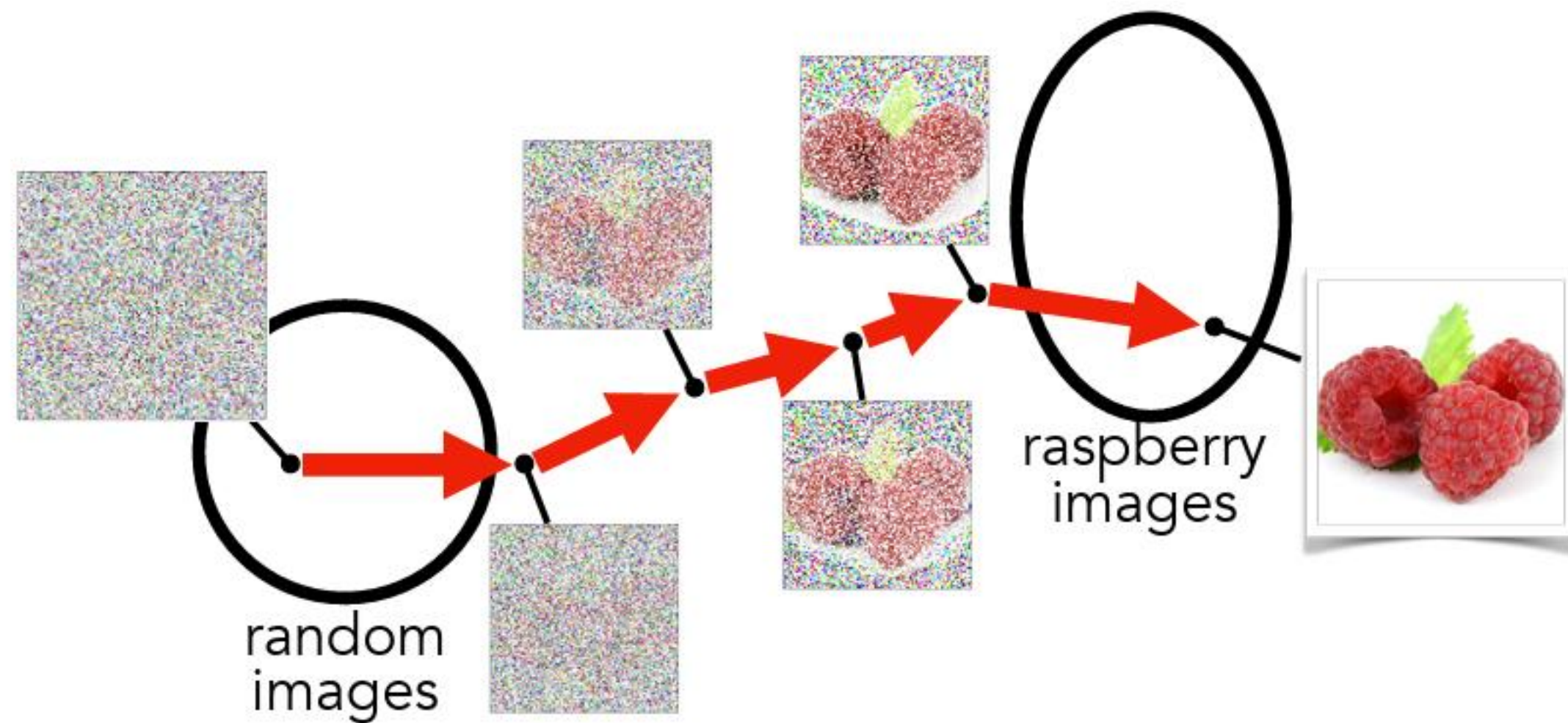
Recover structure from noise

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$

$$p(x_{t-1}|x_t, \mathbf{w}) = \mathcal{N}(x_{t-1}|\mu(x_t, \mathbf{w}, t), \beta_t \mathbf{I})$$



Diffusion : Reverse Process (Generation)



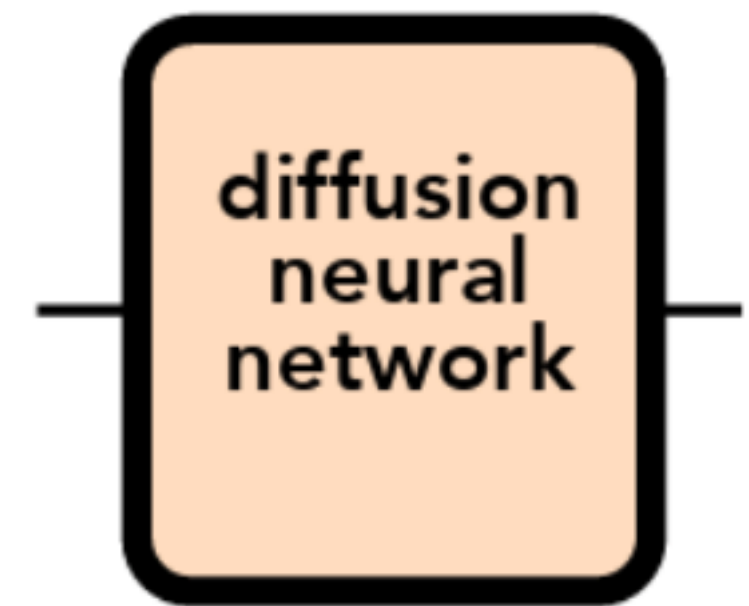
Recover structure from noise

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1})}{q(\mathbf{x}_t)}$$

$$p(x_{t-1}|x_t, \mathbf{w}) = \mathcal{N}(x_{t-1}|\mu(x_t, \mathbf{w}, t), \beta_t \mathbf{I})$$

Trains a single neural network to learn

$$\mu(x_t, \mathbf{w}, t)$$



What the Model Actually Learns

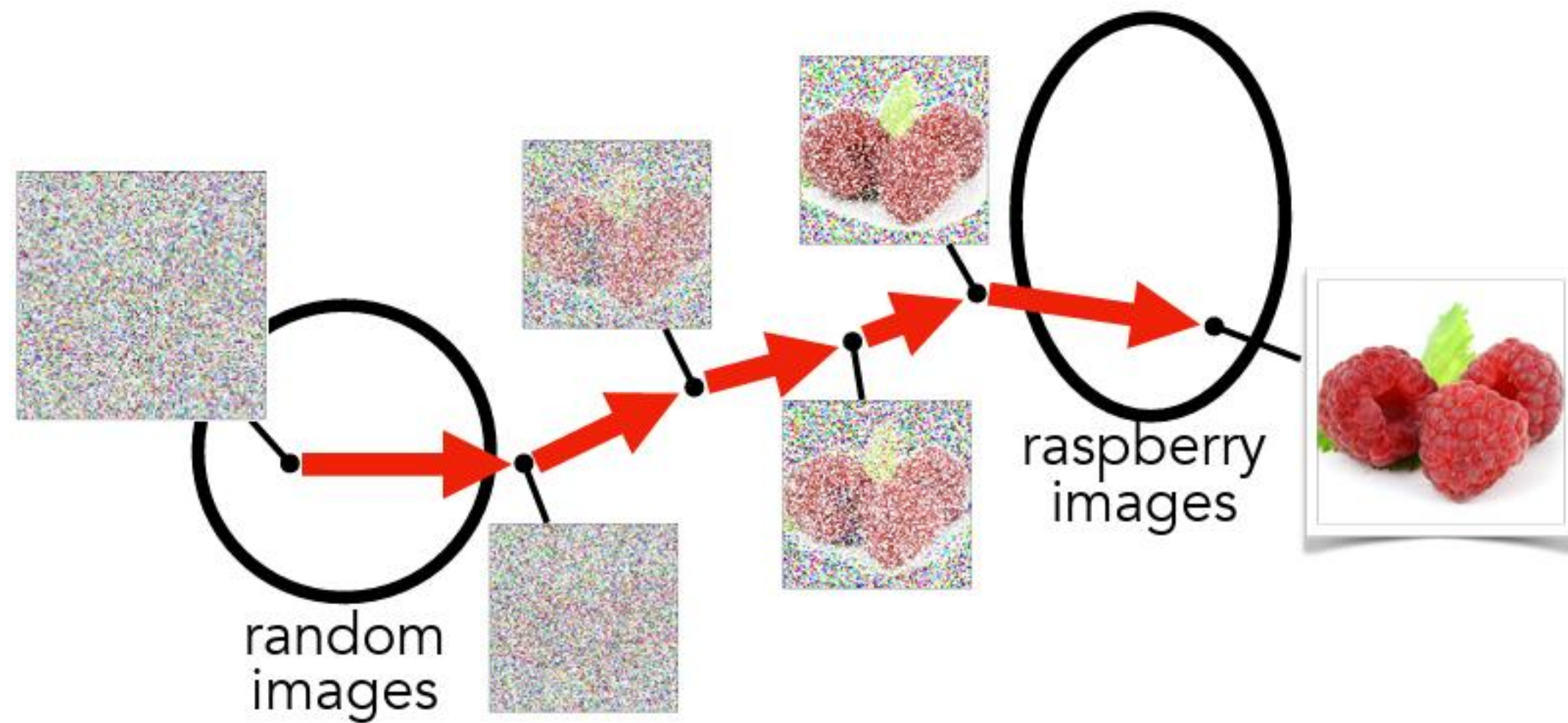
Instead of directly predicting x_{t-1} , we train a model to predict the noise added at step t

$$\epsilon_{\theta}(x_t, t)$$

Training objective

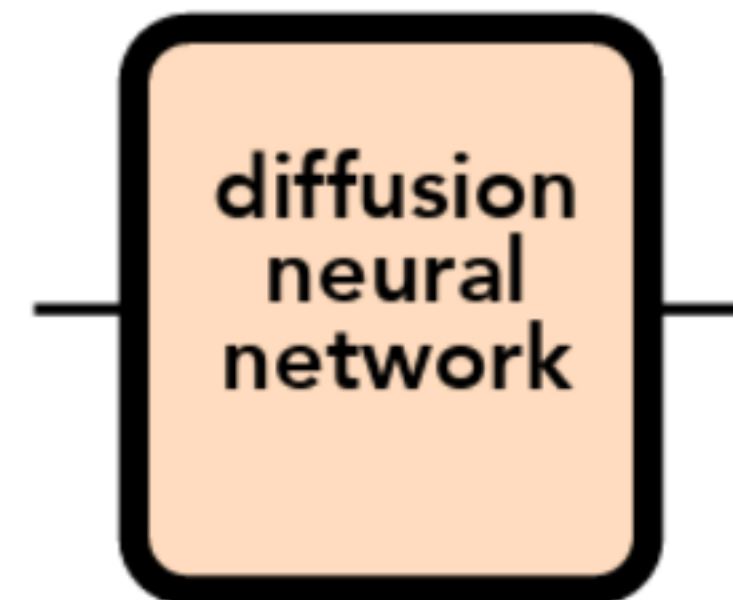
$$\mathcal{L} = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_{\theta}(x_t, t)\|^2]$$

Diffusion : Reverse Process (Generation)



recover structure from noise
Trains a neural network to learn

$$p_{\theta}(x_{t-1}|x_t)$$



Sampling (Generation) Procedure

1. Start from

$$x_T \sim \mathcal{N}(0, I)$$

2. Iteratively apply

$$x_{t-1} = \text{denoise}(x_t)$$

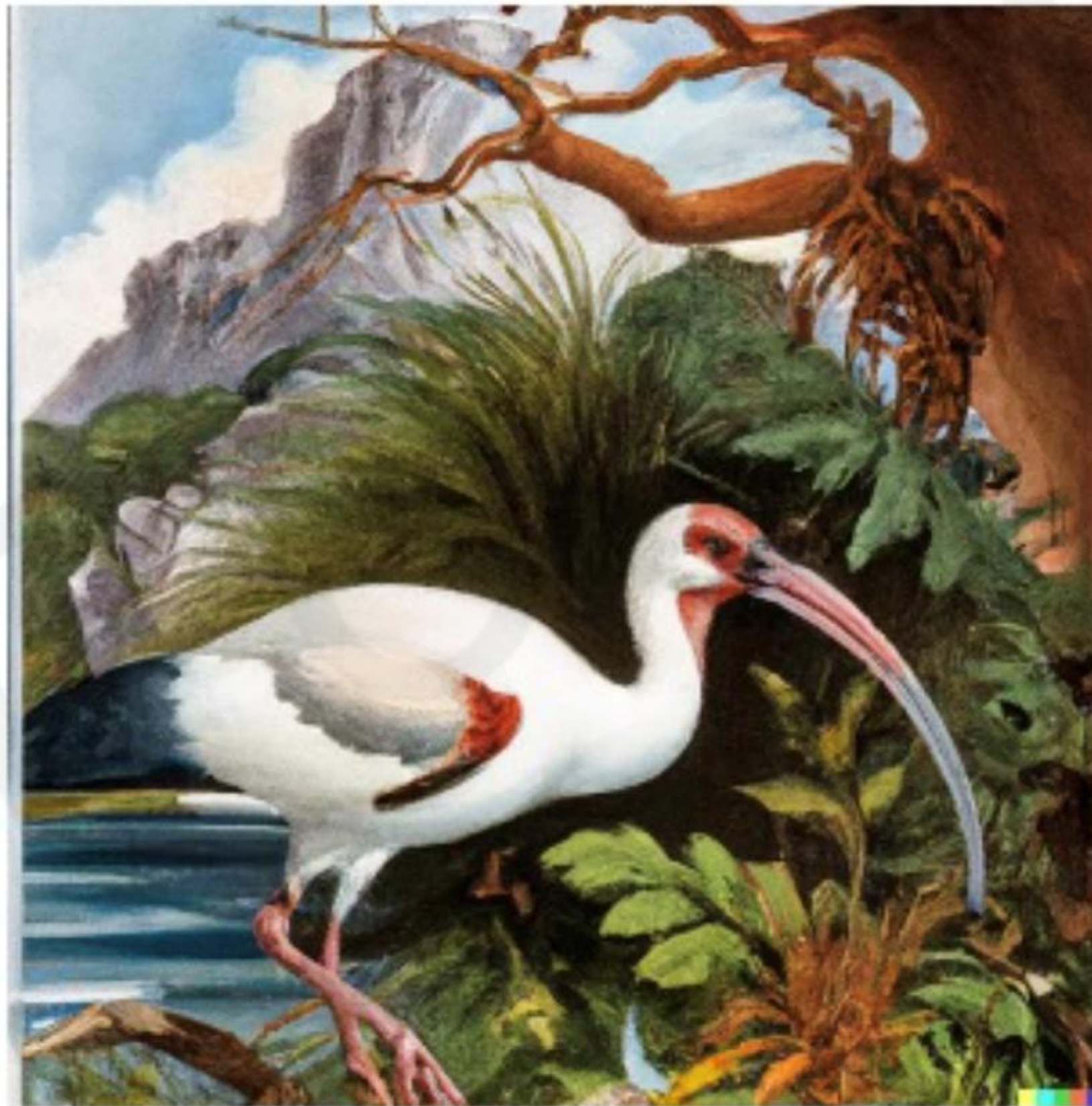
3. After several iterations, real image gets generated

Generating images from languages

“a painting of a fox sitting in a field at sunrise in the style of Claude Monet”



“an ibis in the wild, painted in the style of John Audubon”



“close-up of a snow leopard in the snow hunting, rack focus, nature photography”



Improvements in [transformer](#)-based [deep neural networks](#), particularly large language models (LLMs), enabled an [AI boom](#) of generative AI systems in the early 2020s. These include [chatbots](#) such as [ChatGPT](#), [Copilot](#), [Gemini](#) and [LLaMA](#), [text-to-image artificial intelligence image generation](#) systems such as [Stable Diffusion](#), [Midjourney](#) and [DALL-E](#), and [text-to-video](#) AI generators such as [Sora](#)

Companies such as [OpenAI](#), [Anthropic](#), [Microsoft](#), [Google](#), and [Baidu](#) as well as numerous smaller firms have developed generative AI models. However, concerns have been raised about the potential misuse of generative AI such as [cybercrime](#), the use of [fake news](#) or [deepfakes](#) to deceive or manipulate people, and the mass replacement of human jobs.



OpenAI closes Sora video-making app and cancels \$1bn Disney deal

3 days ago

Share  Save 

Osmond Chia, business reporter and Emma Calder, technology reporter

References

- [1] Hanna Peters, Norma Cueto Calis, “An empirical comparison of generative capabilities of GAN vs VAE”, KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science
- [2] Diederik P. Kingma and Max Welling (2019), “An Introduction to Variational Autoencoders”, Foundations and Trends in Machine Learning.
- [3] NIPS 2016 Tutorial: “Generative Adversarial Networks”, Ian Goodfellow
- [4] Ian J. Goodfellow, Jean Pouget-Abadie* , Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair[†] , Aaron Courville, Yoshua Bengio, “Generative Adversarial Nets”, Département d’informatique et de recherche opérationnelle ´ Université de Montreal
- [5] Ian Goodfellow and Yoshua Bengio and Aaron Courville, “Deep Learning”, MIT Press Book.